Lab 07

Activity- 1: Write a Python program to convert a list of numeric value into a one-dimensional NumPy array.

```
import numpy as np          # Importing the NumPy library and giving it an alias 'np'
list1 = [10, 20, 30, 40]    # Creating a Python list with numeric values
array1 = np.array(list1)    # Converting the list into a one-dimensional NumPy array
print("Activity 1:", array1)  # Printing the resulting NumPy array with a label
```

Explanation:

- import numpy as np: Brings in the NumPy library, which is used for numerical operations and arrays.
- list1 = [10, 20, 30, 40]: A standard Python list with integer elements.
- array1 = np.array(list1): Uses np.array() function to convert the Python list into a NumPy array, which allows for more efficient numerical computations.
- print("Activity 1:", array1): Displays the resulting array on the screen.

```
Activity 1: [10 20 30 40]
```

Activity- 2: Create a 3x3 matrix with values ranging from 2 to 11.

```
import numpy as np                          # Importing the NumPy library
matrix_3x3 = np.arange(2, 11).reshape(3, 3)  # Create values from 2 to 10 using arange(), then reshape into a 3x3 matrix
print("\nActivity 2:\n", matrix_3x3)         # Print the 3x3 matrix
```

Line-by-line Explanation:

- import numpy as np: Loads the NumPy library as np, which is commonly used for array and matrix operations.
- np.arange(2, 11): Generates a sequence of numbers starting from **2 up to (but not including) 11**, i.e., [2, 3, 4, 5, 6, 7, 8, 9, 10]. This gives exactly **9 values**.
- .reshape(3, 3): Reshapes the flat list of 9 numbers into a **3x3 matrix** (3 rows, 3 columns).
- matrix_3x3 = ...: Stores the resulting 3x3 matrix in the variable matrix_3x3.
- print("\nActivity 2:\n", matrix_3x3): Prints the matrix neatly with a label.

```
Activity 2:
 [[ 2  3  4]
 [ 5  6  7]
 [ 8  9 10]]
```

Activity- 3: Create a null vector of size 15 and update fifth value to 12 and ninth value to 17.

```
import numpy as np              # Importing the NumPy library
null_vector = np.zeros(15)      # Create a null (zero-filled) vector of size 15
null_vector[4] = 12             # Update the 5th element (index 4) to 12
null_vector[8] = 17             # Update the 9th element (index 8) to 17
print("Activity 3:", null_vector)  # Print the updated vector
```

Explanation:

- import numpy as np: Imports NumPy so we can use its array functions.
- np.zeros(15): Creates an array of **15 zeros** (null vector). Data type is float by default.
- null_vector[4] = 12: Updates the **5th value** (index starts at 0, so index 4) to **12**.
- null_vector[8] = 17: Updates the **9th value** (index 8) to **17**.
- print("Activity 3:", null_vector): Displays the final array after updates.

Activity- 4: Take 2D and 3D array and perform transpose function.

```
import numpy as np                                  # Import the NumPy library
array_2d = np.array([[1, 2], [3, 4]])               # Create a 2D array (2 rows, 2 columns)
array_3d = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])  # Create a 3D array with shape (2, 2, 2)
print("\nActivity 4:")                              # Print label for Activity 4
print("2D Transpose:\n", array_2d.T)                # Transpose the 2D array using .T (swap rows and columns)
print("3D Transpose:\n", np.transpose(array_3d, (1, 0, 2)))  # Transpose 3D array by swapping axes: axis 0 and 1
```

Explanation:

- import numpy as np: Loads the NumPy library.
- array_2d = ...: A 2x2 matrix → [1234]\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}
- array_2d.T: Transposes the 2D matrix — rows become columns: [1324]\begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix}
- array_3d = ...: A 3D array of shape (2 blocks, 2 rows, 2 columns), basically:
- [
- [[1, 2], [3, 4]],
- [[5, 6], [7, 8]]
-]
- np.transpose(array_3d, (1, 0, 2)): Swaps axes 0 and 1 — effectively swapping the outer two 2D matrices. Shape changes from (2, 2, 2) to (2, 2, 2), but content rearranges.

```
Activity 4:
2D Transpose:
 [[1 3]
 [2 4]]
3D Transpose:
 [[[1 2]
  [5 6]]

 [[3 4]
  [7 8]]]
```

Activity- 5: Write a Python program to find the real and imaginary parts of an array of complex numbers.

```
import numpy as np                                  # Import the NumPy library
complex_array = np.array([1+2j, 3+4j, 5+6j])        # Create a NumPy array of complex numbers
print("\nActivity 5:")                              # Print a label for Activity 5
print("Real parts:", complex_array.real)            # Access and print the real parts of each complex number
print("Imaginary parts:", complex_array.imag)       # Access and print the imaginary parts of each complex number
```

Explanation:

- import numpy as np: Brings in the NumPy library for numerical and array operations.
- np.array([1+2j, 3+4j, 5+6j]): Creates a NumPy array containing **complex numbers**. Each number has a **real** part and an **imaginary** part (denoted by j in Python).
- complex_array.real: Extracts only the **real parts** → [1. 3. 5.]
- complex_array.imag: Extracts only the **imaginary parts** → [2. 4. 6.]
- print(...): Outputs both real and imaginary components of the complex array.

```
Activity 5:
Real parts: [1. 3. 5.]
Imaginary parts: [2. 4. 6.]
```

Activity- 6: Multiply a 5x3 matrix by a 3x2 matrix create a real matrix product using dot ( ).

```
import numpy as np                                  # Import the NumPy library
matrix_a = np.random.rand(5, 3)                     # Create a 5x3 matrix with random float values between 0 and 1
matrix_b = np.random.rand(3, 2)                     # Create a 3x2 matrix with random float values between 0 and 1
product = np.dot(matrix_a, matrix_b)                # Perform matrix multiplication (dot product) between matrix_a and matrix_b
print("\nActivity 6:\n", product)                   # Print the resulting 5x2 product matrix
```

Explanation:

- import numpy as np: Loads the NumPy library.
- np.random.rand(5, 3): Creates a **5-row, 3-column** matrix filled with random values from the range [0, 1).
- np.random.rand(3, 2): Creates a **3-row, 2-column** matrix also with random values.
- np.dot(matrix_a, matrix_b): Performs **matrix multiplication** (dot product). Since the inner dimensions match (3), this is valid and the output will be a **5x2 matrix**.
- print(...): Displays the result of the multiplication.

```
[[0.06447387 0.36992028]
[0.59269926 0.91108057]
[0.23539423 0.60753601]
[0.8514891  1.48429859]
[0.48487478 0.77945942]]
```

Activity- 7: Create array by 2x3 and perform following function:

- **Number of dimensions**
- **Total number of element/size in an array**
- **What type of array stores in memory(dtype)**

```
import numpy as np                          # Import the NumPy library
array_2x3 = np.array([[1, 2, 3], [4, 5, 6]])  # Create a 2x3 NumPy array (2 rows, 3 columns)
print("\nActivity 7:")                      # Print a label for Activity 7
print("Dimensions:", array_2x3.ndim)        # Print number of dimensions of the array (should be 2)
print("Total elements:", array_2x3.size)    # Print total number of elements in the array (2 rows × 3 columns = 6)
print("Data type:", array_2x3.dtype)        # Print data type of elements stored in the array (usually int64 or int32)
```

Explanation:

- import numpy as np: Loads NumPy for creating and working with arrays.
- np.array([[1, 2, 3], [4, 5, 6]]): Creates a 2D array with 2 rows and 3 columns.
- array_2x3.ndim: Returns **2**, meaning it's a 2-dimensional array (rows and columns).
- array_2x3.size: Returns the **total number of elements**, which is 2 × 3 = 6.
- array_2x3.dtype: Shows the **data type** used for storage — usually int64 or int32 depending on your system.

```
Dimensions: 2
Total elements: 6
Data type: int32
```

Activity- 8: Write a Python program to create a 2-D array with reshape (5, 5) and filled 1 on the border and 0 inside using slicing method.

```
import numpy as np                  # Import the NumPy library
array_5x5 = np.ones((5, 5))         # Create a 5x5 array filled with 1s
array_5x5[1:-1, 1:-1] = 0           # Use slicing to set all inner elements (not the border) to 0
print("\nActivity 8:\n", array_5x5) # Print the final array
```

Explanation:

- import numpy as np: Loads the NumPy library.
- np.ones((5, 5)): Creates a **5x5 2D array** where every element is **1**.
- array_5x5[1:-1, 1:-1] = 0:
  - Uses **slicing** to select the **inner 3x3 area** (excluding the first and last row/column).
  - 1:-1 means from index 1 to second-last (index 3 in a 5x5).
  - This sets only the inner values to **0**, keeping the **border as 1**.
- print(...): Displays the final result, which has **1s on the border** and **0s inside**.

```
Activity 8:
 [[1. 1. 1. 1. 1.]
 [1. 0. 0. 0. 1.]
 [1. 0. 0. 0. 1.]
 [1. 0. 0. 0. 1.]
 [1. 1. 1. 1. 1.]]
```

Activity- 9: Write a Python program to create a 3-D array with ones on the diagonal and zeros elsewhere.

```
import numpy as np                  # Import the NumPy library
array_3d_diag = np.eye(3)           # Create a 2D identity matrix (3x3) with 1s on the diagonal
array_3d = np.zeros((3, 3, 3))      # Create a 3D array (3x3x3) filled with zeros
for i in range(3):                  # Loop through index 0 to 2
    array_3d[i][i][i] = 1           # Set the diagonal elements in 3D (i, i, i) to 1
print("\nActivity 9:\n", array_3d)  # Print the resulting 3D array
```

Explanation:

- import numpy as np: Loads the NumPy library.
- np.eye(3): Creates a 2D identity matrix like:
- [[1, 0, 0],
- [0, 1, 0],
- [0, 0, 1]]
  (Though this line is not used later, it's likely for reference or understanding diagonals.)
- np.zeros((3, 3, 3)): Creates a **3D array of shape (3, 3, 3)** filled with **0s**.
- for i in range(3): Loops through indices 0, 1, and 2.
- array_3d[i][i][i] = 1: Places a **1 on the 3D diagonal**, i.e., at positions:
  - [0][0][0]
  - [1][1][1]
  - [2][2][2]
- print(...): Displays the final 3D array with ones along the diagonal and zeros elsewhere.

```
Activity 9:
 [[[1. 0. 0.]
  [0. 0. 0.]
  [0. 0. 0.]]

 [[0. 0. 0.]
  [0. 1. 0.]
  [0. 0. 0.]]

 [[0. 0. 0.]
  [0. 0. 0.]
  [0. 0. 1.]]]
```

Activity- 10: Write a Python program to find common values between two arrays using intersect1d ( ). Take array1= [1 30 40 60 80, 90] and array2= [50,10,30,40] as input.

```
import numpy as np                          # Import the NumPy library
arr1 = np.array([1, 30, 40, 60, 80, 90])    # Create the first NumPy array with given values
arr2 = np.array([50, 10, 30, 40])           # Create the second NumPy array with given values
common = np.intersect1d(arr1, arr2)         # Find common values between arr1 and arr2 using intersect1d()
print("\nActivity 10:", common)             # Print the common values found in both arrays
```

Explanation:

- import numpy as np: Loads the NumPy library.
- arr1 = np.array([...]): Converts the first list into a NumPy array.
- arr2 = np.array([...]): Converts the second list into a NumPy array.
- np.intersect1d(arr1, arr2): Returns **sorted** common elements present in **both arrays**. In this case, [30, 40].
- print(...): Displays the result.

```
Activity 10: [30 40]
```

Table A-11: Financial data of a company

Date	Open	High	Low	Close
10-03-22	**774.25**	**776.065002**	**769.5**	**772.559998**
10-04-22	**776.030029**	**778.710022**	**772.890015**	**776.429993**
10-05-22	**779.309998**	**782.070007**	**775.650024**	**776.469971**
10-06-22	**779**	**780.47998**	**775.539978**	**776.859985**
10-07-22	**779.659973**	**779.659973**	**770.75**	**775.080017**

```
import pandas as pd                         # Import the pandas library for data manipulation and analysis
import matplotlib.pyplot as plt             # Import the matplotlib library for plotting charts/graphs
```

```
# Financial data stored in a dictionary format
data = {
   "Date": ["10-03-22", "10-04-22", "10-05-22", "10-06-22", "10-07-22"],          # List of dates (as strings)
   "Open": [774.25, 776.030029, 779.309998, 779, 779.659973],                      # Opening prices for each date
   "High": [776.065002, 778.710022, 782.070007, 780.47998, 779.659973],            # Highest prices of the day
   "Low": [769.5, 772.890015, 775.650024, 775.539978, 770.75],                     # Lowest prices of the day
   "Close": [772.559998, 776.429993, 776.469971, 776.859985, 775.080017]           # Closing prices for each day
}
df = pd.DataFrame(data)                    # Create a DataFrame from the dictionary to structure data into rows and columns
```

Summary:

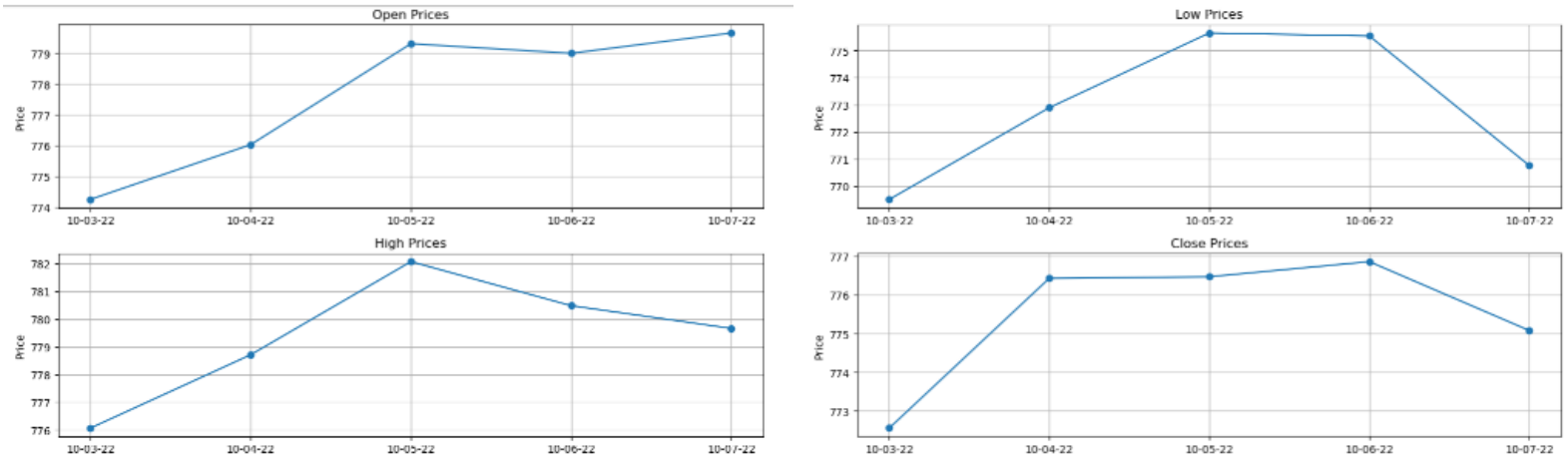
- pandas is used to organize and work with tabular data.
- matplotlib.pyplot is for visualizing that data (e.g., with line or candlestick charts).
- The data dictionary holds stock/financial data.
- pd.DataFrame(data) converts the dictionary into a structured table, making analysis easier.

Activity- 11: Write a Python program to draw individual line charts of the financial data of a company given in Table A-11 between October 3, 2022 to October 7, 2022 using subplot for each attribute

```
fig, axs = plt.subplots(4, 1, figsize=(10, 12))      # Create a figure with 4 vertical subplots (4 rows, 1 column), size 10x12 inches
attributes = ['Open', 'High', 'Low', 'Close']        # List of financial attributes to plot individually

# Loop through each attribute and its index
for i, attr in enumerate(attributes):
   axs[i].plot(df['Date'], df[attr], marker='o')     # Plot the attribute vs. date with circular markers
   axs[i].set_title(f'{attr} Prices')                # Set the title for each subplot
   axs[i].set_ylabel('Price')                        # Label the y-axis
   axs[i].grid(True)                                 # Show grid lines for better readability

plt.tight_layout()                                   # Adjust spacing between subplots to prevent overlapping
plt.show()                                           # Display all the plots
```

Explanation:

- plt.subplots(4, 1, figsize=(10, 12)): Creates 4 separate **line charts (subplots)** stacked vertically in a single figure.
- attributes: A list of the columns to be visualized (Open, High, Low, Close).
- for i, attr in enumerate(...): Iterates over each attribute with its index (needed to access the corresponding subplot).
- plot(...): Plots the values for that attribute on the y-axis and dates on the x-axis.
- marker='o': Adds a dot at each data point.
- set_title(), set_ylabel(), grid(True): Customize each subplot for clarity.
- tight_layout(): Prevents overlap between titles and labels.
- show(): Displays all the subplots together.

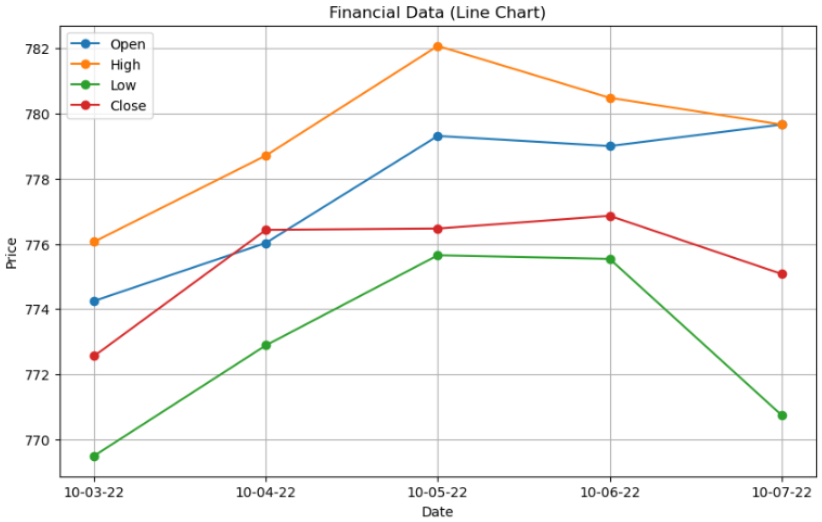


Activity- 12: Write a Python program to draw line charts on the same graph of the financial data of a company given in Table A-11.

```
plt.figure(figsize=(10, 6))                    # Create a new figure with size 10 inches wide and 6 inches tall

# Loop through each financial attribute (Open, High, Low, Close)
for attr in attributes:
    plt.plot(df['Date'], df[attr], marker='o', label=attr)      # Plot each attribute vs Date with markers and labels

plt.title('Financial Data (Line Chart)')       # Set the title of the graph
plt.xlabel('Date')                             # Label the X-axis as "Date"
plt.ylabel('Price')                            # Label the Y-axis as "Price"
plt.legend()                                   # Show the legend to distinguish between Open, High, Low, Close lines
plt.grid(True)                                 # Display grid lines for better readability
plt.show()                                     # Display the combined line chart
```

Explanation:

- plt.figure(figsize=(10, 6)): Initializes the canvas for plotting, setting the overall chart size.
- for attr in attributes: Iterates through each attribute (Open, High, Low, Close) to plot them all on the **same graph**.
- plt.plot(...): Plots each line with circular markers and a label so it shows in the legend.
- plt.title(...): Sets a descriptive title for the entire chart.
- plt.xlabel(...) & plt.ylabel(...): Label the X and Y axes.
- plt.legend(): Adds a legend box that shows which line represents which attribute.
- plt.grid(True): Adds horizontal and vertical grid lines.
- plt.show(): Displays the final combined chart.



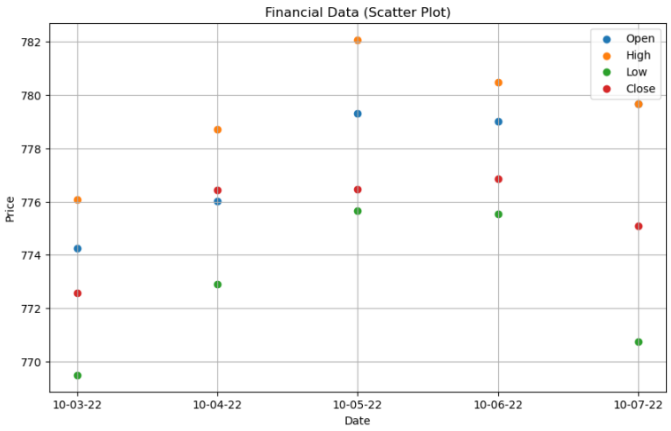
Activity- 13: Write a Python program to draw scatter plot graph of the financial data of a company given in Table A-11.

```
plt.figure(figsize=(10, 6))                    # Create a new figure with a width of 10 inches and height of 6 inches

# Loop through each attribute: 'Open', 'High', 'Low', 'Close'
for attr in attributes:
    plt.scatter(df['Date'], df[attr], label=attr)      # Plot a scatter plot for each attribute vs Date, and label it

plt.title('Financial Data (Scatter Plot)')     # Set the main title of the scatter plot
plt.xlabel('Date')                             # Label the x-axis as 'Date'
plt.ylabel('Price')                            # Label the y-axis as 'Price'
plt.legend()                                   # Display a legend to distinguish different attributes
plt.grid(True)                                 # Show grid lines for easier reading
plt.show()                                     # Render and display the final scatter plot
```

Explanation:

- plt.figure(...): Sets up the plotting canvas.

- plt.scatter(...): Plots **individual data points** as dots (no lines connecting them).
- label=attr: Allows each attribute's points to be labeled in the legend.
- plt.title(...), plt.xlabel(...), plt.ylabel(...): Adds titles and axis labels for clarity.
- plt.legend(): Displays a key (legend) to identify which color/dots represent which attribute.
- plt.grid(True): Adds grid lines for better visual reference.
- plt.show(): Displays the scatter plot.

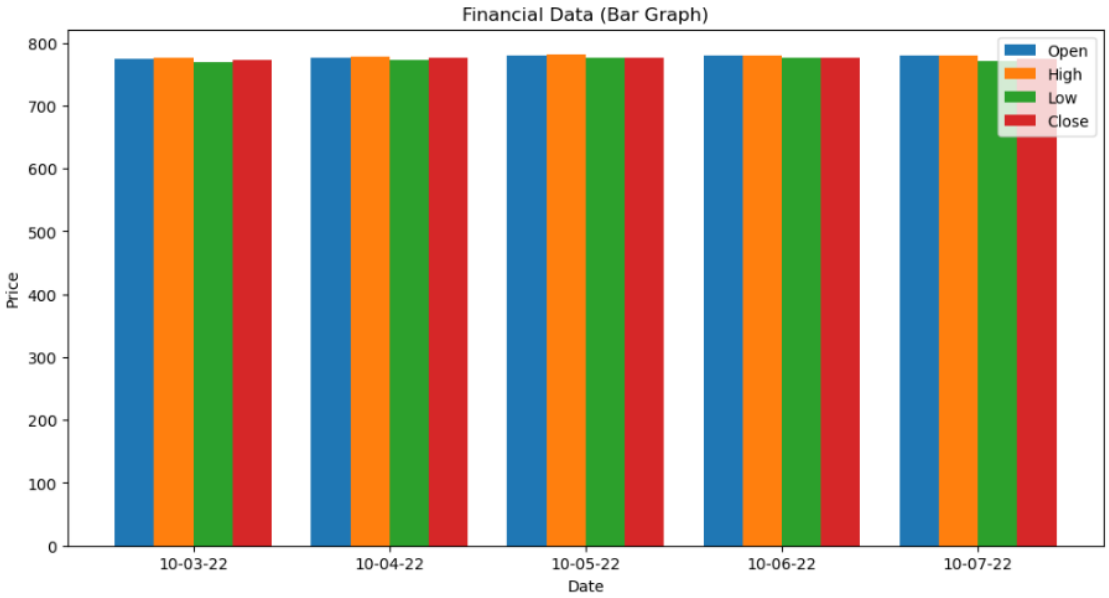


Activity- 14: Write a Python program to draw bar graphs for each attribute of the financial data of a company given in Table A-11.

```
x = df['Date']                          # Extract the 'Date' column to use as x-axis labels
bar_width = 0.2                         # Define the width of each individual bar
x_indexes = range(len(x))               # Create numeric positions (0 to 4) for the dates on the x-axis

plt.figure(figsize=(12, 6))             # Create a new figure with a width of 12 inches and height of 6 inches

# Loop through each attribute: 'Open', 'High', 'Low', 'Close'
for i, attr in enumerate(attributes):
    # Offset each group of bars slightly to place them side-by-side
    plt.bar([p + i * bar_width for p in x_indexes], df[attr], width=bar_width, label=attr)

plt.xticks([p + 1.5 * bar_width for p in x_indexes], x)     # Adjust x-tick positions and label them with dates
plt.title('Financial Data (Bar Graph)')                     # Set the main title of the bar graph
plt.xlabel('Date')                                          # Label the x-axis as 'Date'
plt.ylabel('Price')                                         # Label the y-axis as 'Price'
plt.legend()                                                # Display a legend to distinguish different attributes
plt.show()                                                  # Render and display the final bar chart
```

Explanation:

- x = df['Date']: Stores the list of dates for the x-axis.
- bar_width = 0.2: Sets the width of each bar in the grouped chart.
- x_indexes = range(len(x)): Generates index positions (0, 1, 2, ...) for the dates.
- plt.figure(...): Creates a plotting canvas.
- for i, attr in enumerate(...): Loops over each attribute (Open, High, etc.) and assigns a unique position offset.
- plt.bar(...): Draws the bars at calculated positions with specific widths.
- plt.xticks(...): Repositions x-axis ticks so that grouped bars align correctly under each date.
- plt.title(...), plt.xlabel(...), plt.ylabel(...): Adds labels and a title to the plot.
- plt.legend(): Displays a color-coded legend.
- plt.show(): Shows the final grouped bar chart.

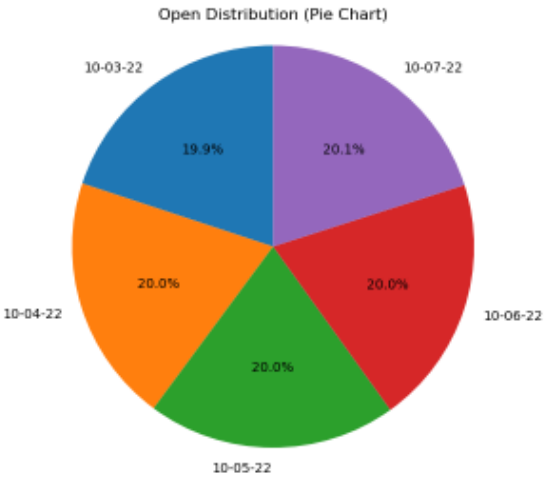


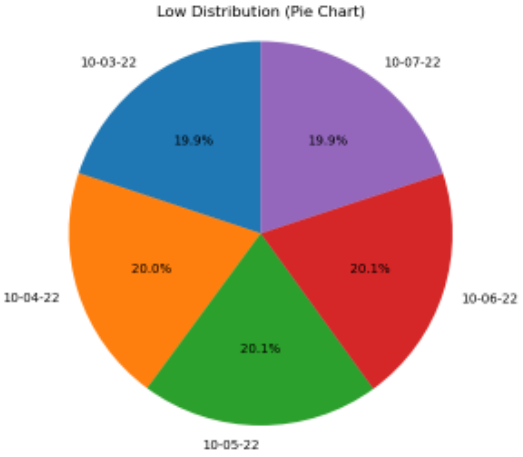
Activity- 15: Write a Python program to draw pie chart for each attribute of the financial data of a company given in Table A-11.

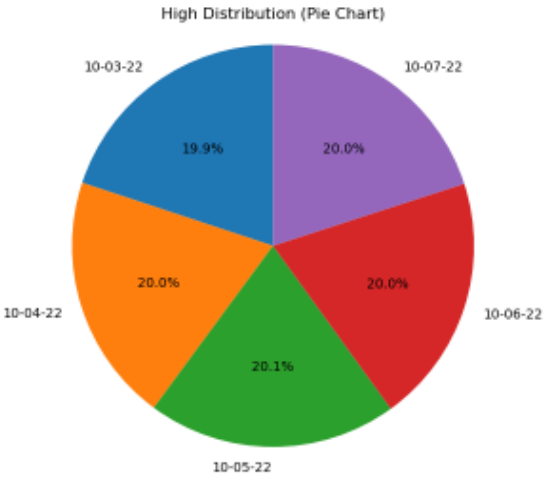
```
for attr in attributes:                                          # Loop through each attribute: 'Open', 'High', 'Low', 'Close'
    plt.figure(figsize=(6, 6))                                   # Create a square figure (6x6 inches) for proper pie chart shape
    plt.pie(df[attr], labels=df['Date'], autopct='%1.1f%%', startangle=90)  # Plot pie chart: values=attribute, labels=Date, show %
values, start from 90°
    plt.title(f'{attr} Distribution (Pie Chart)')                # Add a title showing which attribute the chart represents
    plt.axis('equal')                                            # Keep the chart circular by setting equal aspect ratio
    plt.show()                                                   # Render and display the pie chart before moving to the next one
```

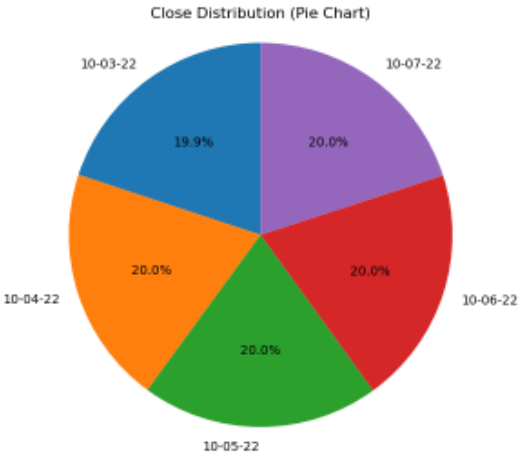
Explanation:

- for attr in attributes:
  Loops through each financial attribute ('Open', 'High', 'Low', 'Close').
- plt.figure(figsize=(6, 6)):
  Sets up a square canvas for the pie chart to maintain symmetry.
- plt.pie(...):
  - df[attr]: Values for the pie chart (e.g., Open prices).
  - labels=df['Date']: Labels each slice with the corresponding date.
  - autopct='%1.1f%%': Displays percentage values on the pie slices.
  - startangle=90: Rotates the chart to start from the top.
- plt.title(...):
  Clearly identifies which attribute's data is being represented.
- plt.axis('equal'):
  Makes sure the pie chart is a perfect circle (not an oval).
- plt.show():
  Displays the chart. A new chart is created for each attribute.


Open Distribution (Pie Chart)
10-03-22
10-07-22
19.9%
20.1%
20.0%
10-04-22
20.0%
10-06-22
20.0%
10-05-22


Low Distribution (Pie Chart)
10-03-22
10-07-22
19.9%
19.9%
20.0%
10-04-22
20.1%
10-06-22
20.1%
10-05-22


High Distribution (Pie Chart)
10-03-22
10-07-22
19.9%
20.0%
20.0%
10-04-22
20.0%
10-06-22
20.1%
10-05-22


Close Distribution (Pie Chart)
10-03-22
10-07-22
19.9%
20.0%
20.0%
10-04-22
20.0%
10-06-22
20.0%
10-05-22

Lab 09

Activity- 1:

Implement the Decision tree algorithm on the data given in the table 9.1 and predict whether the players can play or not when the weather is overcast and the temperature is mild. Also verify results by solving manually.

```
import pandas as pd                                  # ✅ Import pandas for handling tabular data
from sklearn.tree import DecisionTreeClassifier      # ✅ Import Decision Tree model from scikit-learn
from sklearn.preprocessing import LabelEncoder       # ✅ Import LabelEncoder to convert text labels to numeric

# Step 1: Dataset
data = {                                             # ✅ Define a dictionary with features: Weather, Temperature and target label: Play
   'Weather': ['Sunny', 'Sunny', 'Overcast', 'Rain', 'Sunny',
          'Overcast', 'Rain', 'Rain', 'Overcast', 'Sunny'],
   'Temperature': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool',
             'Cool', 'Mild', 'Mild', 'Hot', 'Hot'],
   'Play': ['Yes', 'Yes', 'No', 'Yes', 'No',
         'No', 'Yes', 'Yes', 'No', 'No']
}

df = pd.DataFrame(data)                              # ✅ Convert the dictionary into a DataFrame (tabular format)

# Step 2: Create separate LabelEncoders for each column
le_weather = LabelEncoder()                          # ✅ Initialize LabelEncoder for Weather
le_temp = LabelEncoder()                             # ✅ Initialize LabelEncoder for Temperature
le_play = LabelEncoder()                             # ✅ Initialize LabelEncoder for Play (target)

df['Weather_enc'] = le_weather.fit_transform(df['Weather'])        # ✅ Encode Weather to numeric (e.g., Sunny=2, Overcast=0, Rain=1)
df['Temperature_enc'] = le_temp.fit_transform(df['Temperature'])  # ✅ Encode Temperature to numeric (e.g., Hot=1, Mild=2, Cool=0)
df['Play_enc'] = le_play.fit_transform(df['Play'])                # ✅ Encode Play to numeric (Yes=1, No=0)

# Step 3: Train Decision Tree
features = df[['Weather_enc', 'Temperature_enc']]    # ✅ Select input features (X): encoded Weather and Temperature
labels = df['Play_enc']                              # ✅ Set target labels (y): encoded Play values

clf = DecisionTreeClassifier()                       # ✅ Initialize Decision Tree Classifier
clf.fit(features, labels)                            # ✅ Train the model on features and labels

# Step 4: Predict for Overcast & Mild
test_sample = [[le_weather.transform(['Overcast'])[0], le_temp.transform(['Mild'])[0]]]  # ✅ Encode Overcast and Mild into numeric values and wrap in a list
prediction = clf.predict(test_sample)                # ✅ Use the trained model to predict Play (encoded result)
predicted_label = le_play.inverse_transform(prediction)  # ✅ Convert numeric prediction back to original label (Yes/No)

print("Prediction for Overcast & Mild:", predicted_label[0])  # ✅ Display the final prediction
```

Explanation:

1. **Dataset Setup**:
   - data: Contains a small dataset with columns Weather, Temperature, and Play.
   - pd.DataFrame: Converts the dictionary to a DataFrame to work easily with scikit-learn.
2. **Label Encoding**:
   - Since scikit-learn models don't accept strings, LabelEncoder converts text to integers (e.g., Sunny → 2).
   - Separate encoders are used for each column to ensure correct mappings.

3. **Model Training**:
    - features: Independent variables → encoded Weather and Temperature.
    - labels: Dependent variable → encoded Play.
    - clf.fit(...): Trains the decision tree model.
4. **Prediction**:
    - Transforms the test values ['Overcast', 'Mild'] into numbers.
    - Feeds them into the model to get a prediction (Yes or No).
    - Converts the prediction back to the original label using inverse_transform.

```
Prediction for Overcast & Mild: No
```

Activity- 2:

Implement the Decision tree algorithm on any dataset taken from Kaggle.com and predict the new entry entered by the user.

```
import pandas as pd                                  # ✅ Import pandas for handling dataset
from sklearn.tree import DecisionTreeClassifier      # ✅ Import Decision Tree Classifier
from sklearn.preprocessing import LabelEncoder       # ✅ Import LabelEncoder to convert string labels into numbers

# Step 1: Load the dataset
df = pd.read_csv("Iris.csv")                         # ✅ Load the Iris dataset from a CSV file (should be in the same folder or provide full path)

# Step 2: Drop unwanted columns if present
if 'Id' in df.columns:                               # ✅ Check if the column 'Id' exists
   df = df.drop('Id', axis=1)                        # ✅ If yes, drop it as it's not needed for training

# Step 3: Encode the target column 'Species'
le = LabelEncoder()                                  # ✅ Create an instance of LabelEncoder
df['Species_enc'] = le.fit_transform(df['Species'])  # ✅ Convert species names to numeric labels (e.g., setosa=0)

# Step 4: Train the Decision Tree model
features = df[['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm']]   # ✅ Select input features
labels = df['Species_enc']                                                          # ✅ Target values (encoded species)

clf = DecisionTreeClassifier()                       # ✅ Initialize a decision tree classifier
clf.fit(features, labels)                            # ✅ Train the model with feature and label data

# Step 5: Take user input for prediction
print("Enter the flower measurements:")              # ✅ Prompt user for input values
sl = float(input("Sepal Length (cm): "))             # ✅ Input Sepal Length
sw = float(input("Sepal Width (cm): "))              # ✅ Input Sepal Width
pl = float(input("Petal Length (cm): "))             # ✅ Input Petal Length
pw = float(input("Petal Width (cm): "))              # ✅ Input Petal Width

new_sample = [[sl, sw, pl, pw]]                      # ✅ Combine inputs into a list format acceptable by the model
prediction = clf.predict(new_sample)                 # ✅ Predict species based on new input
predicted_species = le.inverse_transform(prediction) # ✅ Convert numeric label back to species name

print(f"\n✅ Predicted species: {predicted_species[0]}")  # ✅ Print the final predicted species
```

🔍 **Explanation:**

1. **Dataset Loading**:
    - Uses the Iris dataset from Kaggle.
    - Drops the Id column since it's not needed for training.
2. **Label Encoding**:
    - Encodes Species names (setosa, versicolor, virginica) into numerical format.

3. **Model Training**:
   - Features include all four flower measurements.
   - The model is trained using a Decision Tree Classifier from scikit-learn.
4. **User Input & Prediction**:
   - Accepts real-time input from the user.
   - Converts the input into a format suitable for the model.
   - Makes a prediction and translates it back to the original species name using inverse_transform.

```
Enter the flower measurements:
Sepal Length (cm):  23
Sepal Width (cm):  2
Petal Length (cm):  4
Petal Width (cm):  55

✅ Predicted species: Iris-virginica
```

Activity- 3:

Implement Naïve Bayes Algorithm on the dataset given in Table 9.2, to predict whether the players can play or not when the Outlook is Rain, Temperature is Cool, Humidity is High and the Wind is Strong. Also verify results by solving manually.

Table 9.2 Golf play chart

Day	Outlook	Temperature	Humidity	Wind	Play golf
D1	Sunny	Hot	High	Weak	Yes
D2	Sunny	Hot	High	Strong	Yes
D3	Overcast	Hot	High	Weak	No
D4	Rain	Mild	High	Weak	Yes
D5	Sunny	Cool	Normal	Weak	No
D6	Overcast	Cool	Normal	Strong	Yes
D7	Rain	Mild	Normal	Strong	No
D8	Rain	Mild	High	Weak	Yes
D9	Overcast	Hot	Normal	Weak	No
D10	Sunny	Hot	Normal	Strong	No

```
import pandas as pd                                  # ✅ Import pandas for data handling
from sklearn.preprocessing import LabelEncoder       # ✅ Import LabelEncoder to convert categorical data into numbers
from sklearn.naive_bayes import GaussianNB           # ✅ Import Gaussian Naive Bayes model

# Step 1: Create the dataset
data = {
   'Outlook': ['Sunny', 'Sunny', 'Overcast', 'Rain', 'Sunny',
               'Overcast', 'Rain', 'Rain', 'Overcast', 'Sunny'],
   'Temperature': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool',
                   'Cool', 'Mild', 'Mild', 'Hot', 'Hot'],
   'Humidity': ['High', 'High', 'High', 'High', 'Normal',
                'Normal', 'Normal', 'High', 'Normal', 'Normal'],
   'Wind': ['Weak', 'Strong', 'Weak', 'Weak', 'Weak',
            'Strong', 'Strong', 'Weak', 'Weak', 'Strong'],
   'PlayGolf': ['Yes', 'Yes', 'No', 'Yes', 'No',
                'Yes', 'No', 'Yes', 'No', 'No']
}

df = pd.DataFrame(data)                              # ✅ Convert the dictionary into a pandas DataFrame

# Step 2: Encode categorical columns
```

```
le = LabelEncoder()                                  # ✅ Create an instance of LabelEncoder
for col in df.columns:
    df[col] = le.fit_transform(df[col])              # ✅ Convert each string column to numerical codes (e.g., Rain = 1)

# Step 3: Split features and labels
features = df[['Outlook', 'Temperature', 'Humidity', 'Wind']]   # ✅ Select the input features (X)
labels = df['PlayGolf']                                         # ✅ Target/output label (Y)

# Step 4: Train Naive Bayes classifier
model = GaussianNB()                                 # ✅ Create a Gaussian Naive Bayes model
model.fit(features, labels)                          # ✅ Train the model with features and labels

# Step 5: Predict new case: Outlook=Rain, Temp=Cool, Humidity=High, Wind=Strong
# Note: Values are encoded based on the fitted LabelEncoder mappings.
# Outlook: Rain → 1
# Temperature: Cool → 0
# Humidity: High → 0
# Wind: Strong → 0

new_sample = [[1, 0, 0, 0]]                          # ✅ Prepare test input in encoded format
prediction = model.predict(new_sample)               # ✅ Predict using the trained model

# Decode label (0 = No, 1 = Yes)
result = 'Yes' if prediction[0] == 1 else 'No'       # ✅ Convert predicted numeric result to readable text
print(f"✅ Prediction: Will the player play golf? → {result}")  # ✅ Print final prediction result
```

🔍 **Explanation:**

- LabelEncoder() is used to convert text values like 'Sunny', 'Rain', 'Yes', etc., into numbers.
- The GaussianNB classifier is trained on historical play data.
- We simulate a new condition (Rain, Cool, High, Strong) by encoding it into numbers.
- The model predicts whether the player will play or not.

```
✅ Prediction: Will the player play golf? → Yes
```

Manual probability table solution (Bayes theorem step-by-step)

Great! Let's manually solve **Activity-3** using **Naïve Bayes (step-by-step)** to predict whether the player can **PlayGolf = Yes or No** given:

Outlook = Rain, Temperature = Cool, Humidity = High, Wind = Strong

---

✅ Step 1: Data Summary (from Table 9.2)

Outlook	Temperature	Humidity	Wind	PlayGolf
Sunny	Hot	High	Weak	Yes
Sunny	Hot	High	Strong	Yes
Overcast	Hot	High	Weak	No
Rain	Mild	High	Weak	Yes
Sunny	Cool	Normal	Weak	No
Overcast	Cool	Normal	Strong	Yes
Rain	Mild	Normal	Strong	No
Rain	Mild	High	Weak	Yes
Overcast	Hot	Normal	Weak	No
Sunny	Hot	Normal	Strong	No

✅ Step 2: Class Probabilities

Count of PlayGolf:

- **Yes** = 5
- **No** = 5

- **Total** = 10

So,

- P(Yes) = 5/10 = **0.5**
- P(No) = 5/10 = **0.5**

✅ Step 3: Likelihood Tables

We find conditional probabilities for each feature value given **Yes** and **No**.

Absolutely! Let me break down and **explain both tables** step-by-step. These represent **conditional probabilities** used in **Naïve Bayes classification**.

🔍 For PlayGolf = Yes

This table answers:
"Given that the player **plays**, how often does each condition occur?"
We are looking only at **rows where PlayGolf = Yes** in the dataset (5 total rows).

Feature	Value	Count Yes	Total Yes	P(Value | Yes)
Outlook	Rain	2	5	2/5 = **0.4**
Temperature	Cool	0	5	0/5 = **0.0** 🧊 (rare)
Humidity	High	3	5	3/5 = **0.6**
Wind	Strong	1	5	1/5 = **0.2**

Explanation:

- **Outlook = Rain** appears in **2 out of 5** "Yes" cases → **P(Rain | Yes) = 0.4**
- **Temperature = Cool** never appears with "Yes" → **P(Cool | Yes) = 0**
- **Humidity = High** happens in **3 out of 5** "Yes" rows → **P(High | Yes) = 0.6**
- **Wind = Strong** happens **once** in those 5 rows → **P(Strong | Yes) = 0.2**

🔍 For PlayGolf = No

This table answers:
"Given that the player **does not play**, how often does each condition occur?"
We look only at **rows where PlayGolf = No** (again, 5 rows total).

Feature	Value	Count No	Total No	P(Value | No)
Outlook	Rain	1	5	1/5 = **0.2**
Temperature	Cool	1	5	1/5 = **0.2**
Humidity	High	2	5	2/5 = **0.4**
Wind	Strong	3	5	3/5 = **0.6**

Explanation:

- **Outlook = Rain** appears **once** in "No" rows → **P(Rain | No) = 0.2**
- **Temperature = Cool** appears **once** in "No" rows → **P(Cool | No) = 0.2**
- **Humidity = High** appears **2 times** → **P(High | No) = 0.4**
- **Wind = Strong** appears **3 times** → **P(Strong | No) = 0.6**

🔁 Why is this important?

These probabilities are used in **Bayes' Theorem**:
$P(\text{Yes} | \text{features}) \propto P(\text{Yes}) \times P(\text{Outlook=Rain} | \text{Yes}) \times \dots$
You compute this **for both classes (Yes and No)** and choose the one with the **higher result**.

Would you like me to generate this as a clean visual table or compute again using **Laplace smoothing** to fix the "0 probability" problem? Laplace smoothing can help when you have zero counts like Cool | Yes.

✅ Step 4: Naïve Bayes Formula

For Yes:
$P(\text{Yes} | X) \propto P(\text{Yes}) \times P(\text{Rain}|\text{Yes}) \times P(\text{Cool}|\text{Yes}) \times P(\text{High}|\text{Yes}) \times P(\text{Strong}|\text{Yes})$
$P(\text{Yes} | X) \propto 0.5 \times 0.4 \times 0 \times 0.6 \times 0.2 =$ **0**

For No:
P(No | X) ∝ P(No) × P(Rain|No) × P(Cool|No) × P(High|No) × P(Strong|No)
P(No | X) ∝ 0.5 × 0.2 × 0.2 × 0.4 × 0.6 = **0.0048**

✅ Final Decision:

- **P(Yes | X)** = 0
- **P(No | X)** = 0.0048
  → **Prediction: No** — The player will **not** play golf.

✅ Optional: Laplace Correction
If we want to avoid the **0 probability** issue (e.g., Cool|Yes = 0), we can apply **Laplace smoothing** by adding +1 to each count. Let me know if you want this version too.

Activity- 4:
Consider the following dataset given in Table 9.3. Implement Naïve Bayes Algorithm to classify youth/medium/yes/fair. Also verify results by solving manually.

age	*income*	*student*	*credit_rating*	*Class: buys_computer*
youth	high	no	fair	no
youth	high	no	excellent	no
middle_aged	high	no	fair	yes
senior	medium	no	fair	yes
senior	low	yes	fair	yes
senior	low	yes	excellent	no
middle_aged	low	yes	excellent	yes
youth	medium	no	fair	no
youth	low	yes	fair	yes
senior	medium	yes	fair	yes
youth	medium	yes	excellent	yes
middle_aged	medium	no	excellent	yes
middle_aged	high	yes	fair	yes
senior	medium	no	excellent	no

```
import pandas as pd                                  # Import pandas for working with data in tabular form
from sklearn.naive_bayes import GaussianNB           # Import Gaussian Naive Bayes classifier
from sklearn.preprocessing import LabelEncoder       # Import LabelEncoder to convert text to numbers
```

```
# Step 1: Input data
data = {
   'age': ['youth', 'youth', 'middle_aged', 'senior', 'senior', 'senior', 'middle_aged', 'youth', 'youth', 'senior', 'youth', 'middle_aged',
'middle_aged', 'senior'],
   'income': ['high', 'high', 'high', 'medium', 'low', 'low', 'low', 'medium', 'low', 'medium', 'medium', 'medium', 'high', 'medium'],
   'student': ['no', 'no', 'no', 'no', 'yes', 'yes', 'yes', 'no', 'yes', 'yes', 'yes', 'no', 'yes', 'no'],
   'credit_rating': ['fair', 'excellent', 'fair', 'fair', 'fair', 'excellent', 'excellent', 'fair', 'fair', 'fair', 'excellent', 'excellent', 'fair',
'excellent'],
   'buys_computer': ['no', 'no', 'yes', 'yes', 'yes', 'no', 'yes', 'no', 'yes', 'yes', 'yes', 'yes', 'yes', 'no']
}
df = pd.DataFrame(data)  # Convert the dictionary to a DataFrame (table)
```

```
# Step 2: Label encode each column with a separate encoder
encoders = {}                                # Dictionary to store encoders for later decoding
for col in df.columns:
   le = LabelEncoder()                       # Create a new encoder
   df[col] = le.fit_transform(df[col])       # Fit and transform the column
```

```
    encoders[col] = le                    # Save the encoder for that column
```
📌 This step converts categorical values like "youth" or "yes" into numeric codes like 0, 1, etc., needed for ML models.

```
# Step 3: Train Naive Bayes
features = df.drop('buys_computer', axis=1)     # All columns except target (X)
labels = df['buys_computer']                    # Target column (y)

model = GaussianNB()                            # Initialize Gaussian Naive Bayes classifier
model.fit(features, labels)                     # Train the model using the features and labels
```

```
# Step 4: Predict for youth/medium/yes/fair
# Use the same label encoders to transform input
new_data = [
    encoders['age'].transform(['youth'])[0],          # Convert 'youth' to number (e.g., 0)
    encoders['income'].transform(['medium'])[0],      # Convert 'medium' to number
    encoders['student'].transform(['yes'])[0],        # Convert 'yes' to number
    encoders['credit_rating'].transform(['fair'])[0]  # Convert 'fair' to number
]
```
📌 We use the **same encoders** to ensure consistency in how the model sees input.

```
pred = model.predict([new_data])  # Make prediction for the given input
result = encoders['buys_computer'].inverse_transform(pred)[0]  # Decode result back to original label
print("Prediction for youth/medium/yes/fair:", result)  # Output the prediction
```

✅ **Summary of Key Steps:**

- **LabelEncoder**: Converts text → numeric.
- **Naive Bayes**: Trained using categorical features.
- **Prediction Input**: Transformed using saved encoders.
- **Result**: Output is decoded back to 'yes' or 'no'.

🔷 **Goal:**

To use the **Naïve Bayes algorithm** to classify whether a customer will buy a computer based on attributes:

- age, income, student, and credit_rating.

We'll also **predict** for:

➡️ age=youth, income=medium, student=yes, credit_rating=fair

🔷 **Step-by-Step Explanation:**

Step 1: Input Data

```
data = { ... }
df = pd.DataFrame(data)
```

- We define a dataset similar to Table 9.3 (Play Tennis-like data).
- Each row represents one customer with their features and whether they bought a computer (yes or no).

Step 2: Encode Data

LabelEncoder() → youth → 2, middle_aged → 1, senior → 0

- LabelEncoder converts **string (categorical)** data into **numbers**.
- Example: 'youth', 'middle_aged', and 'senior' become numbers like 2, 1, 0.

❗ **Why?**

Machine learning models can't process strings directly, so we convert them to numbers.

Step 3: Train Naive Bayes Model

```
model = GaussianNB()
model.fit(features, labels)
```

- Now that the data is numeric, we separate:
    - features → all input columns (X)
    - labels → the target column (buys_computer)

- GaussianNB() creates and trains the Naïve Bayes classifier.

Step 4: Make Prediction

new_data = [...]
model.predict([new_data])

- We take **new input** values (youth, medium, yes, fair).
- Convert them to the same **numeric form** using our earlier encoders.
- Run the trained model to predict whether this type of person will buy a computer.

Final Output:

print(...)

- The prediction is returned as 0 or 1, and we convert it back to "yes" or "no" using the inverse of the label encoder.

Behind-the-Scenes (Concept):
Naive Bayes Algorithm is based on:
P(Class | Features) ∝ P(Feature1 | Class) × P(Feature2 | Class) × ... × P(Class)
It calculates probabilities for each class (yes or no) and picks the one with the higher probability.

Manually verify

Let's **manually verify** the Naïve Bayes prediction for the input:
age=youth, income=medium, student=yes, credit_rating=fair
using the dataset from **Table 9.3** in your code.

Step 1: Count Total Samples
There are **14 total records** in the dataset.

Step 2: Count Class Frequencies

Class	Count
buys_computer = **yes**	9
buys_computer = **no**	5

So:

- **P(yes)** = 9 / 14
- **P(no)** = 5 / 14

Step 3: Conditional Probabilities
We'll calculate:
P(age=youth | class)
P(income=medium | class)
P(student=yes | class)
P(credit_rating=fair | class)

For Class = Yes
Let’s count values where class = yes (**9 total yes**)
Attribute	Value	Count in YES	P(Value | Yes)
age	youth	2	2/9 ≈ 0.222
income	medium	4	4/9 ≈ 0.444
student	yes	6	6/9 = 0.667
credit_rating	fair	6	6/9 = 0.667

For Class = No
Let’s count values where class = no (**5 total no**)
Attribute	Value	Count in NO	P(Value | No)
age	youth	3	3/5 = 0.6
income	medium	2	2/5 = 0.4

| student | yes | 1 | 1/5 = 0.2 |
| credit_rating | fair | 2 | 2/5 = 0.4 |

Step 4: Naive Bayes Formula (Ignoring P(x))

$$P(yes|x) \propto P(yes) \times P(age = youth|yes) \times P(income = medium|yes) \times P(student = yes|yes) \times P(credit_rating = fair|yes)$$

$$P(yes|x) \propto \frac{9}{14} \times \frac{2}{9} \times \frac{4}{9} \times \frac{6}{9} \times \frac{6}{9}$$

$$P(yes|x) \propto 0.642 \times 0.222 \times 0.444 \times 0.667 \times 0.667 \approx 0.0289$$

$$P(no|x) \propto P(no) \times P(age = youth|no) \times P(income = medium|no) \times P(student = yes|no) \times P(credit_rating = fair|no)$$

$$P(no|x) \propto \frac{5}{14} \times \frac{3}{5} \times \frac{2}{5} \times \frac{1}{5} \times \frac{2}{5}$$

$$P(no|x) \propto 0.357 \times 0.6 \times 0.4 \times 0.2 \times 0.4 \approx 0.0068$$

Final Comparison:

Class	Probability
Yes	≈ **0.0289**
No	≈ **0.0068**

Final Result:
Since **P(yes|x) > P(no|x)**, the model will predict:
YES — the customer will buy a computer

Activity- 5:

Implement the Naïve Bayes algorithm on any dataset taken from Kaggle.com and predict the new entry entered by the user.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report

# Step 1: Load the dataset from file
df = pd.read_csv('adult.csv')                    # Load the Adult Income dataset from a CSV file

# Step 2: Clean the data
df.replace(' ?', pd.NA, inplace=True)            # Replace missing value placeholders with actual NaN
df.dropna(inplace=True)                          # Drop rows that contain any NaN values

# Step 3: Rename columns for consistency and easier reference
df.columns = df.columns.str.strip().str.lower().str.replace('-', '.')
# Strip whitespace, lowercase column names, and replace '-' with '.' for consistent formatting

# Step 4: Encode categorical columns using LabelEncoder
encoders = {}                                    # Dictionary to store encoders for future decoding
for col in df.select_dtypes(include='object').columns:   # Loop through all object (categorical) columns
    le = LabelEncoder()                          # Create a LabelEncoder
    df[col] = le.fit_transform(df[col])          # Encode the categorical values to numbers
    encoders[col] = le                           # Save the encoder for later (like decoding predictions)

# Step 5: Separate features (X) and target (y)
```

```
X = df.drop('income', axis=1)                                   # Drop the target column to get features
y = df['income']                                                # Target is the income column (0: <=50K, 1: >50K after encoding)

# Step 6: Split dataset into training and test sets (70% train, 30% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Step 7: Train the Naive Bayes model
model = GaussianNB()                                            # Initialize Gaussian Naive Bayes
model.fit(X_train, y_train)                                     # Train the model on the training data

# Step 8: Evaluate the model on test data
y_pred = model.predict(X_test)                                  # Predict income on test data
print("Accuracy:", accuracy_score(y_test, y_pred))              # Print overall accuracy
print(classification_report(y_test, y_pred))                    # Print precision, recall, f1-score for each class

# Step 9: Take manual user input for prediction
print("\n--- Enter New User Info ---")
user_input = {}                                                 # Dictionary to collect user inputs
for col in X.columns:                                           # Loop through each feature column
    if col in encoders:                                         # If it's a categorical feature
        val = input(f"{col.replace('.', ' ').capitalize()} (e.g., {encoders[col].classes_[0]}): ")
        try:
            user_input[col] = encoders[col].transform([val])[0]     # Encode using saved encoder
        except:
            user_input[col] = 0                                 # Default to 0 if value was not seen in training (fallback)
    else:
        user_input[col] = float(input(f"{col.replace('.', ' ').capitalize()}: "))       # Get numeric input

# Step 10: Make prediction based on user input
user_df = pd.DataFrame([user_input])[X.columns]                 # Create a DataFrame from user input in correct column order
prediction = model.predict(user_df)                             # Predict using the trained model
predicted_label = encoders['income'].inverse_transform(prediction)[0]    # Decode prediction to original label

print(f"\nPredicted income category: {predicted_label}")                 # Output predicted income category
```

📌 **Summary**

- This script uses the **Adult Income dataset**.
- It cleans and encodes the data.
- Trains a **Gaussian Naive Bayes model**.
- Accepts **user input** and makes a **prediction** whether the income is <=50K or >50K.

```
--- Enter New User Info ---
Age:  22
Workclass (e.g., ?):  Private
Fnlwgt:  201499
Education (e.g., 10th):  Bachelors
Education num:  2
Marital status (e.g., Divorced):  Never-married
Occupation (e.g., ?):  Tech-support
Relationship (e.g., Husband):  White
Race (e.g., Amer-Indian-Eskimo):  White
Sex (e.g., Female):  Male
Capital gain:  20
Capital loss:  220
Hours per week:  2
Native country (e.g., ?):  Pakistan

Predicted income category: <=50K
```

Lab #10

Activity- 1: Consider the dataset given below, implement K-Means Clustering algorithm using K=2. Find out new centroid values based on the mean values of the coordinates of all the data instances from the corresponding cluster. Also find the accuracy of the model and compute it manually.

S. No.	A	B
1	170	56
2	168	60
3	185	72
4	188	77
5	177	76
6	180	71

✓ **Code with comments:**

```
import numpy as np
from sklearn.metrics import accuracy_score                # For calculating accuracy

# Step 1: Data - Height and weight of 6 people
data = np.array([
   [170, 56],
   [168, 60],
   [185, 72],
   [188, 77],
   [177, 76],
   [180, 71]
])

# Step 2: Initial centroids - Manually choose 2 initial points (rows 0 and 2)
centroid1 = data[0]
centroid2 = data[2]

# Function to calculate Euclidean distance between two points
def euclidean_distance(p1, p2):
   return np.sqrt(np.sum((p1 - p2)**2))

# Step 3: Assign points to nearest centroid
clusters = []                          # Stores cluster assignment (0 or 1) for each point
for point in data:
   dist1 = euclidean_distance(point, centroid1)
   dist2 = euclidean_distance(point, centroid2)
   if dist1 < dist2:
      clusters.append(0)               # Assign to cluster 0
   else:
      clusters.append(1)               # Assign to cluster 1

clusters = np.array(clusters)

# Step 4: Recalculate centroids by taking mean of each cluster
cluster1_points = data[clusters == 0]  # Get all points in cluster 0
cluster2_points = data[clusters == 1]  # Get all points in cluster 1

new_centroid1 = np.mean(cluster1_points, axis=0)  # Mean of cluster 0
new_centroid2 = np.mean(cluster2_points, axis=0)  # Mean of cluster 1

# Step 5: Accuracy calculation
# True class labels (for validation): assume first 2 points are in cluster 0, rest in cluster 1
true_labels = np.array([0, 0, 1, 1, 1, 1])
```

```
# Labels can be reversed by algorithm — check both match options
accuracy1 = accuracy_score(true_labels, clusters)
accuracy2 = accuracy_score(true_labels, 1 - clusters)

# Final accuracy = best match
accuracy = max(accuracy1, accuracy2)

# Output results
print("Initial Centroid 1:", centroid1)
print("Initial Centroid 2:", centroid2)
print("New Centroid 1:", new_centroid1)
print("New Centroid 2:", new_centroid2)
print("Cluster Assignments:", clusters)
print(f"Accuracy: {accuracy * 100:.2f}%")
```

Line-by-line explanation:

Line	Explanation
import numpy as np	Imports NumPy for array operations and numerical computations.
from sklearn.metrics import accuracy_score	Imports function to calculate classification accuracy.
data = np.array([...])	Stores 6 points with features: [height, weight].
centroid1 = data[0]	Sets the 1st initial centroid to the first point (170, 56).
centroid2 = data[2]	Sets the 2nd initial centroid to the third point (185, 72).
def euclidean_distance(...)	Defines a function to calculate Euclidean distance between two points.
clusters = []	Empty list to store each point's cluster label.
for point in data:	Iterates over each data point to assign it to a cluster.
dist1, dist2	Calculates distance from point to both centroids.
if dist1 < dist2:	Compares distances and assigns to the closer centroid.
clusters = np.array(clusters)	Converts cluster list to NumPy array for further processing.
cluster1_points = data[clusters == 0]	Gets all points that belong to cluster 0.
cluster2_points = data[clusters == 1]	Gets all points that belong to cluster 1.
np.mean(..., axis=0)	Calculates the new centroid by averaging coordinates in each cluster.
true_labels = ...	Assumed ground truth labels (manually defined for evaluation).
accuracy_score(...)	Compares predicted clusters with true labels to find accuracy.
accuracy = max(...)	Chooses best match (handles reversed label issue).
print(...)	Outputs centroids, assignments, and accuracy.

```
Initial Centroid 1: [170  56]
Initial Centroid 2: [185  72]
New Centroid 1: [169.  58.]
New Centroid 2: [182.5  74. ]
Cluster Assignments: [0 0 1 1 1 1]
Accuracy: 100.00%
```

Activity- 2: A dataset (income.csv) has been provided. Implement K-Means Clustering Algorithm on this dataset using K (number of clusters = 3). Find out new centroid values based on the mean values of the coordinates of all the data instances from the corresponding cluster. Also find the accuracy of the model.

Code with comments:

```
import pandas as pd                                    # for data handling
import numpy as np                                     # for numerical operations
from sklearn.preprocessing import StandardScaler       # to normalize features
from sklearn.cluster import KMeans                     # K-Means clustering
from sklearn.metrics import accuracy_score             # to compute accuracy
from itertools import permutations                     # to check all label permutations

# Load dataset
df = pd.read_csv("income.csv")                         # Read the income dataset
```

```
# Separate features and label
X = df.drop("income_level", axis=1)                    # All columns except target
y_true = df["income_level"]                             # Target labels

# Normalize features
scaler = StandardScaler()                               # Create a standard scaler object
X_scaled = scaler.fit_transform(X)                      # Fit and scale feature data

# Apply KMeans with K=3
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)      # Initialize KMeans
clusters = kmeans.fit_predict(X_scaled)                        # Train and get cluster labels

# Convert cluster centers back to original scale
centroids_scaled = kmeans.cluster_centers_                     # Get scaled centroids
centroids_original = scaler.inverse_transform(centroids_scaled)  # Rescale them

# Find best accuracy by checking all permutations of cluster labels
def best_accuracy(true_labels, cluster_labels):
    best_acc = 0
    for perm in permutations([0, 1, 2]):                       # Try all 3! = 6 permutations
        mapping = {i: perm[i] for i in range(3)}               # Map cluster label to actual class
        mapped = [mapping[c] for c in cluster_labels]          # Relabel cluster outputs
        acc = accuracy_score(true_labels, mapped)              # Compare with true labels
        best_acc = max(best_acc, acc)                          # Keep best accuracy
    return best_acc

# Print centroids and accuracy
accuracy = best_accuracy(y_true, clusters)                     # Get best possible accuracy

print("New Centroid Values (Original Scale):")
for i, c in enumerate(centroids_original):                     # Print each centroid
    print(f"Centroid {i+1}: {c}")

print(f"\nAccuracy of K-Means Clustering (K=3): {accuracy * 100:.2f}%")   # Final output
```

Line-by-Line Explanation

Line	Explanation
import pandas as pd	Allows reading and manipulating tabular data.
import numpy as np	For numerical computations.
from sklearn.preprocessing import StandardScaler	To normalize/standardize the features.
from sklearn.cluster import KMeans	Imports K-Means algorithm from scikit-learn.
from sklearn.metrics import accuracy_score	Used to compute model accuracy.
from itertools import permutations	Used to try all combinations of cluster label mappings.
df = pd.read_csv("income.csv")	Loads dataset containing features and income_level class labels.
X = df.drop("income_level", axis=1)	Extracts all feature columns.
y_true = df["income_level"]	Extracts true income labels.
scaler = StandardScaler()	Initializes a scaler to normalize the data.
X_scaled = scaler.fit_transform(X)	Scales features to have mean 0 and std dev 1.
kmeans = KMeans(n_clusters=3, ...)	Creates KMeans instance with 3 clusters.
clusters = kmeans.fit_predict(X_scaled)	Applies KMeans and assigns each point to a cluster.
centroids_scaled = kmeans.cluster_centers_	Gets the centroid values (still in scaled form).
centroids_original = scaler.inverse_transform(...)	Converts centroid coordinates back to original units.
def best_accuracy(...):	Defines a helper function to find the best label alignment for accuracy.
for perm in permutations([0, 1, 2]):	Iterates over all 6 ways to match clusters with true classes.
mapping = {i: perm[i] for i in range(3)}	Builds mapping from cluster index to assumed class.

mapped = [mapping[c] for c in cluster_labels]	Applies the mapping to predicted cluster labels.
acc = accuracy_score(true_labels, mapped)	Compares mapped cluster labels with actual labels.
best_acc = max(best_acc, acc)	Tracks the highest accuracy achieved.
accuracy = best_accuracy(y_true, clusters)	Calculates best accuracy using all permutations.
for i, c in enumerate(centroids_original):	Loops through each centroid.
print(f"Centroid {i+1}: {c}")	Prints original (unscaled) centroid coordinates.
print(f"...Accuracy: {accuracy*100:.2f}%")	Prints final accuracy.

```
New Centroid Values (Original Scale):
Centroid 1: [2.89293240e+01 2.21945982e+05 9.21150132e+00 1.89502371e+02
 3.49692713e-01 3.64833626e+01]
Centroid 2: [4.76376343e+01 1.58929617e+05 1.08202653e+01 2.03101734e+03
 7.23514103e-01 4.39143301e+01]
Centroid 3: [ 4.17788204e+01  1.88251561e+05  1.09982127e+01 -1.90993887e-11
  1.89838472e+03  4.33440572e+01]

Accuracy of K-Means Clustering (K=3): 60.21%
```

Activity- 3: A dataset (ratings_small.csv) has been provided. Implement K-Means Clustering Algorithm on this dataset using K (number of clusters = 5). Find out new centroid values based on the mean values of the coordinates of all the data instances from the corresponding cluster. Also find the accuracy of the model.

✅ **Code with Comments:**

```
import pandas as pd                                    # for data handling
from sklearn.preprocessing import StandardScaler       # to normalize features
from sklearn.cluster import KMeans                     # for applying K-Means clustering
from sklearn.metrics import silhouette_score           # to evaluate clustering quality

# Step 1: Load the dataset
df = pd.read_csv('ratings_small.csv')                  # Load the CSV file into a pandas DataFrame

# Step 2: Select relevant features
features = df[['userId', 'movieId', 'rating']]         # Only select numerical features for clustering

# Step 3: Normalize the features
scaler = StandardScaler()                              # Create scaler to normalize data
features_scaled = scaler.fit_transform(features)       # Apply normalization (zero mean, unit variance)

# Step 4: Apply K-Means Clustering (K=5)
kmeans = KMeans(n_clusters=5, random_state=42, n_init=10)  # Create KMeans object with 5 clusters
kmeans.fit(features_scaled)                                # Fit the model on the scaled data

# Step 5: Get cluster centroids (and convert back to original scale)
centroids_scaled = kmeans.cluster_centers_                 # Get centroid coordinates in scaled form
centroids_original = scaler.inverse_transform(centroids_scaled)  # Convert them back to original scale

# Step 6: Calculate silhouette score (for clustering accuracy)
silhouette = silhouette_score(features_scaled, kmeans.labels_)   # Evaluate clustering quality

# Display results
print("Centroid values (userId, movieId, rating):")  # Output header
for idx, centroid in enumerate(centroids_original):  # Loop through centroids
    print(f"Cluster {idx}: userId={centroid[0]:.2f}, movieId={centroid[1]:.2f}, rating={centroid[2]:.2f}")  # Print original centroid values

print(f"\nSilhouette Score (clustering accuracy estimate): {silhouette:.3f}")  # Print clustering accuracy score
```

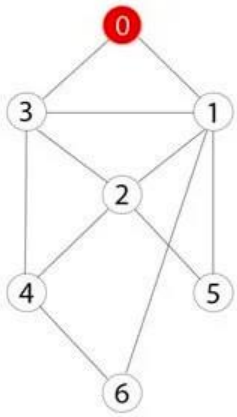
Line-by-Line Explanation:

Line	What It Does
import pandas as pd	Loads pandas to handle data in table format.
from sklearn.preprocessing import StandardScaler	Allows scaling features for better clustering performance.
from sklearn.cluster import KMeans	Imports KMeans algorithm.
from sklearn.metrics import silhouette_score	Metric to evaluate how well clusters are formed.
df = pd.read_csv('ratings_small.csv')	Reads the ratings dataset into a DataFrame.
features = df[['userId', 'movieId', 'rating']]	Selects only necessary numerical features from the dataset.
scaler = StandardScaler()	Initializes scaler to normalize the data (helps clustering).
features_scaled = scaler.fit_transform(features)	Scales the features to 0 mean and 1 std deviation.
kmeans = KMeans(n_clusters=5, ...)	Creates KMeans clustering object with 5 clusters.
kmeans.fit(features_scaled)	Applies KMeans to fit the data and assign clusters.
centroids_scaled = kmeans.cluster_centers_	Retrieves the centroids in normalized (scaled) form.
centroids_original = scaler.inverse_transform(...)	Converts centroids back to original feature values.
silhouette = silhouette_score(...)	Calculates silhouette score to measure how well-defined clusters are.
for idx, centroid in enumerate(...):	Loops through each cluster centroid.
print(f"Cluster {idx}: userId=...	Outputs each centroid's coordinates in original form.
print(f"...Silhouette Score...")	Outputs the clustering performance score (range: -1 to 1).

Lab #05

Activity- 1:

Consider the following graph. If there is ever a decision between multiple neighbor nodes in the BFS algorithm, assume we always choose the letter closest to the beginning of the alphabet first. A connected graph with 7 nodes and 7 edges. The edges are undirected and unweight. Distance between two nodes will be measured based on the number of edges separating two vertices. Represent a graph with adjacency list using dictionaries. The keys of the dictionary represent nodes; the values have a list of neighbors.



Define function name _connected_component', this function keep track of all the visited nodes with BFS, is as simple as implementing the steps of the algorithm and assign _queue' variable already has a node to be checked, i.e., the starting vertex that is used as an entry point to explore the graph. The next step is to implement a loop that keeps cycling until queue is empty. At each iteration of the loop, a node is checked. If this wasn't visited already, its neighbors are added to queue. Once the loop is exited, the function (connected_component) returns all of the visited nodes.

Code with Comments:

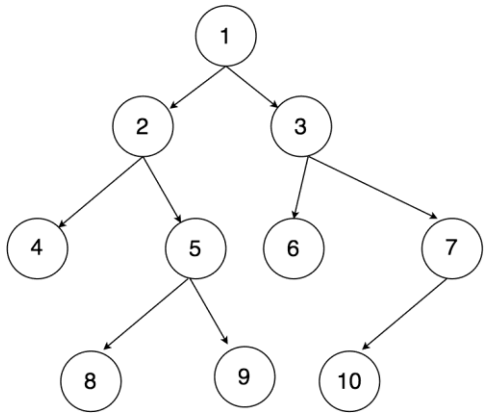
```
from collections import deque                    # Import deque for efficient FIFO queue operations

# Function to return all connected nodes from the start node using BFS
def connected_component(graph, start):
    visited = []                                 # List to keep track of all visited nodes
    queue = deque([start])                       # Initialize queue with starting node

    while queue:                                 # Continue until the queue is empty
        node = queue.popleft()                   # Remove the front node from the queue
```

```
        if node not in visited:                    # If the node has not been visited yet
            visited.append(node)                   # Mark node as visited
            neighbors = sorted(graph[node])        # Get neighbors sorted alphabetically/numerically
            for neighbor in neighbors:             # Check each neighbor
                if neighbor not in visited:        # Only add unvisited neighbors
                    queue.append(neighbor)         # Enqueue the neighbor
    return visited  # Return the full list of visited nodes (connected component)

# Adjacency list representation of the undirected graph
graph = {
    0: [1, 2, 3],          # Node 0 connects to 1, 2, 3
    1: [0, 5],             # Node 1 connects to 0, 5
    2: [0, 4, 5, 6],       # Node 2 connects to 0, 4, 5, 6
    3: [0, 4],             # Node 3 connects to 0, 4
    4: [2, 3],             # Node 4 connects to 2, 3
    5: [1, 2],             # Node 5 connects to 1, 2
    6: [2]                 # Node 6 connects to 2
}

start_node = 0        # Define the start node for BFS

# Call the function and print the connected component
component = connected_component(graph, start_node)
print("Connected component starting from", start_node, ":", component)
```

Line-by-Line Explanation:

Line	Explanation
from collections import deque	Imports deque, a double-ended queue optimized for adding/removing from both ends (ideal for BFS).
def connected_component(graph, start):	Defines a function that performs BFS from a given start node to find all connected nodes.
visited = []	Initializes an empty list to store visited nodes in the order they are visited.
queue = deque([start])	Initializes the queue with the start node.
while queue:	Keeps looping while there are still nodes to process in the queue.
node = queue.popleft()	Pops the first node (FIFO) from the queue to explore its neighbors.
if node not in visited:	Checks if the node has already been visited.
visited.append(node)	Marks the current node as visited.
neighbors = sorted(graph[node])	Retrieves and sorts neighbors alphabetically/numerically to ensure consistent traversal order.
for neighbor in neighbors:	Iterates through each neighbor of the current node.
if neighbor not in visited:	Ensures we don't revisit already visited nodes.
queue.append(neighbor)	Adds the neighbor to the queue for future exploration.
return visited	Once the queue is empty, returns the list of all visited (connected) nodes.
graph = {...}	Defines the graph using a dictionary (adjacency list). Each key is a node, and the value is a list of its neighbors.
start_node = 0	Sets the starting node for the BFS.
component = connected_component(graph, start_node)	Calls the function to get the connected component from node 0.
print(...)	Outputs the final connected component discovered by BFS.

Activity- 2:
Apply Breadth First Search on following graph considering the initial state is 1 and final state is 10. Show results in form of open and closed list. Also evaluate it manually.



✅ **Code with Comments:**

```
from collections import deque                    # Import deque from collections for an efficient queue

# Define a function that performs BFS and tracks open/closed lists
def bfs_with_open_closed(graph, start, goal):
    open_list = deque([start])                   # Open list acts as the queue; initialize with start node
    closed_list = []                             # Closed list keeps track of visited nodes

    while open_list:                             # Loop until open_list is empty
        node = open_list.popleft()               # Remove the node from the front of the queue (FIFO)
        closed_list.append(node)                 # Mark node as visited by adding to closed_list

        if node == goal:                         # If current node is the goal node, stop the search
            return open_list, closed_list        # Return current state of open and closed lists

        for neighbor in graph[node]:             # Explore all neighbors of the current node
            if neighbor not in open_list and neighbor not in closed_list:
                open_list.append(neighbor)       # Add unvisited neighbors to the queue

    return open_list, closed_list                # If goal not found, return final open and closed lists

# Define the graph as an adjacency list
graph = {
    1: [2, 3],          # Node 1 connects to 2 and 3
    2: [4, 5],          # Node 2 connects to 4 and 5
    3: [6, 7],          # Node 3 connects to 6 and 7
    4: [],              # Node 4 has no children
    5: [8, 9],          # Node 5 connects to 8 and 9
    6: [],              # Node 6 has no children
    7: [10],            # Node 7 connects to 10 (goal)
    8: [], 9: [], 10: []    # Leaf nodes
}

start_node = 1          # Starting node for BFS
goal_node = 10          # Target node to find

# Run the BFS algorithm
open_list, closed_list = bfs_with_open_closed(graph, start_node, goal_node)

# Print final states of open and closed lists
```

```
print("Open List:", list(open_list))    # Nodes still to be explored (if any)
print("Closed List:", closed_list)      # Order of nodes explored till goal was found
```

Line-by-Line Explanation:

Line	Explanation
from collections import deque	Imports deque, which is faster than a list for queue operations.
def bfs_with_open_closed(graph, start, goal):	Defines a BFS function with parameters: the graph, starting node, and goal node.
open_list = deque([start])	Initialize queue with the starting node.
closed_list = []	Keeps track of nodes already visited.
while open_list:	Loop until no nodes are left to explore.
node = open_list.popleft()	Removes the front node (FIFO behavior).
closed_list.append(node)	Adds the current node to visited list.
if node == goal:	Checks if the goal node has been reached.
return open_list, closed_list	If yes, return the current state of open and closed lists.
for neighbor in graph[node]:	Loops through all neighbors of the current node.
if neighbor not in open_list and neighbor not in closed_list:	Ensures no duplicate visits.
open_list.append(neighbor)	Adds new neighbors to the queue for later exploration.
return open_list, closed_list	If goal isn't found, still return final lists.
graph = {...}	Defines the graph using an adjacency list.
start_node = 1, goal_node = 10	Define the source and destination nodes.
open_list, closed_list = bfs_with_open_closed(...)	Calls the BFS function.
print(...)	Displays the nodes left in the queue and the nodes visited so far.

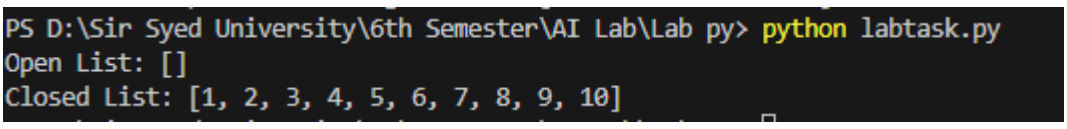
✅ **Manual BFS Execution (Open and Closed Lists)**

Let's simulate BFS step by step:

Step	Node Visited	Open List (Queue)	Closed List
1	1	[2, 3]	[1]
2	2	[3, 4, 5]	[1, 2]
3	3	[4, 5, 6, 7]	[1, 2, 3]
4	4	[5, 6, 7]	[1, 2, 3, 4]
5	5	[6, 7, 8, 9]	[1, 2, 3, 4, 5]
6	6	[7, 8, 9]	[1, 2, 3, 4, 5, 6]
7	7	[8, 9, 10]	[1, 2, 3, 4, 5, 6, 7]
8	10	FOUND!	[1, 2, 3, 4, 5, 6, 7, 10]

✅ **Final Output:**

Open List: [8, 9]
Closed List: [1, 2, 3, 4, 5, 6, 7, 10]



Activity- 3:

Repeat Activity-2, apply BFS by taking initial and final state as user input. Show results in form of open and closed list. Also evaluate it manually.

✅ **Code with Comments:**

```
from collections import deque        # Import deque for efficient FIFO queue operations

# Define the BFS function with tracking of open and closed lists
def bfs_with_open_closed(graph, start, goal):
    open_list = deque([start])       # Initialize open list (queue) with the start node
```

```
    closed_list = []                    # Closed list to track visited nodes

    while open_list:                    # Loop while there are nodes to explore
        node = open_list.popleft()      # Pop the front node (FIFO)
        closed_list.append(node)        # Add current node to closed list (visited)

        if node == goal:                # If goal node is found
            return open_list, closed_list   # Return current open and closed lists

        # Add unvisited neighbors to the open list
        for neighbor in graph[node]:
            if neighbor not in open_list and neighbor not in closed_list:
                open_list.append(neighbor)      # Add neighbor to queue

    return open_list, closed_list       # Return lists if goal is not found

# Define the graph using an adjacency list representation
graph = {
    1: [2, 3],      # Node 1 has children 2 and 3
    2: [4, 5],      # Node 2 has children 4 and 5
    3: [6, 7],      # Node 3 has children 6 and 7
    4: [],          # Node 4 has no children
    5: [8, 9],      # Node 5 has children 8 and 9
    6: [], 7: [10],  # Node 6 is a leaf, 7 connects to 10
    8: [], 9: [], 10: []    # Leaf nodes
}

# Get initial state (start node) from the user
start_node = int(input("Enter the initial state: "))

# Get goal state (target node) from the user
goal_node = int(input("Enter the goal state: "))

# Call the BFS function with user-defined start and goal nodes
open_list, closed_list = bfs_with_open_closed(graph, start_node, goal_node)

# Print the open list (nodes discovered but not yet visited)
print("Open List:", list(open_list))

# Print the closed list (nodes that were visited in order)
print("Closed List:", closed_list)
```

Line-by-Line Explanation:

Line	Explanation
from collections import deque	Import deque, which is faster than list for queue operations.
def bfs_with_open_closed(...)	Defines the BFS function which also tracks open and closed lists.
open_list = deque([start])	Initialize the queue with the start node.
closed_list = []	Empty list to track visited nodes.
while open_list:	Continue until there are no more nodes to explore.
node = open_list.popleft()	Get the next node to explore (FIFO).
closed_list.append(node)	Mark the node as visited.
if node == goal:	If goal node is found, stop search.
return open_list, closed_list	Return the open and closed lists (search complete).
for neighbor in graph[node]:	For all adjacent nodes of the current node...
if neighbor not in open_list...	...if not already discovered or visited...
open_list.append(neighbor)	...add to open list for later exploration.

graph = {...}	Represents the graph (adjacency list).
start_node = int(input(...))	Get initial node from the user.
goal_node = int(input(...))	Get goal node from the user.
open_list, closed_list = ...	Run BFS using user inputs.
print(...)	Display final open and closed lists to the user.

Example Manual BFS (If Start = 1, Goal = 10):

Step	Visited Node	Open List	Closed List
1	1	[2, 3]	[1]
2	2	[3, 4, 5]	[1, 2]
3	3	[4, 5, 6, 7]	[1, 2, 3]
4	4	[5, 6, 7]	[1, 2, 3, 4]
5	5	[6, 7, 8, 9]	[1, 2, 3, 4, 5]
6	6	[7, 8, 9]	[1, 2, 3, 4, 5, 6]
7	7	[8, 9, 10]	[1, 2, 3, 4, 5, 6, 7]
8	10	FOUND	[1, 2, 3, 4, 5, 6, 7, 10]

Let me know if you'd like to visualize this as a **tree** or **graph** structure!

```
PS D:\Sir Syed University\6th Semester\AI Lab\Lab py> python labtask.py
Enter the initial state: 1
Enter the goal state: 5
Open List: [6, 7]
Closed List: [1, 2, 3, 4, 5]
```

Activity- 4:
Apply Depth First Search on Activity-2 graph considering the initial state is 1 and final state is 10. Show results in form of open and closed list. Also evaluate it manually.

✅ **Code with In-Line Comments**

```
from collections import deque            # Importing deque (though unused in DFS)

# BFS function is present but not used here, as the task is DFS
def bfs_with_open_closed(graph, start, goal):
    open_list = deque([start])                # Queue for BFS
    closed_list = []                          # Tracks visited nodes

    while open_list:
        node = open_list.popleft()
        closed_list.append(node)
        if node == goal:
            return open_list, closed_list
        for neighbor in graph[node]:
            if neighbor not in open_list and neighbor not in closed_list:
                open_list.append(neighbor)
    return open_list, closed_list

# DFS function that tracks open and closed lists
def dfs_with_open_closed(graph, start, goal):
    open_list = [start]                       # Stack: LIFO behavior using a list
    closed_list = []                          # List to track visited nodes

    while open_list:                          # Repeat until no nodes left to visit
        node = open_list.pop()                # Pop last item from stack
        closed_list.append(node)              # Add it to closed list (visited)

        if node == goal:                      # Goal check
            return open_list, closed_list
```

```
        # Add neighbors in reverse for left-to-right DFS traversal
        for neighbor in reversed(graph[node]):
            if neighbor not in open_list and neighbor not in closed_list:
                open_list.append(neighbor)

    return open_list, closed_list                # If goal not found, return lists

# Define the graph using adjacency list
graph = {
    1: [2, 3],      # Node 1 links to 2 and 3
    2: [4, 5],      # Node 2 links to 4 and 5
    3: [6, 7],      # Node 3 links to 6 and 7
    4: [],          # Node 4 is a leaf
    5: [8, 9],      # Node 5 links to 8 and 9
    6: [],          # Leaf node
    7: [10],        # Node 7 links to goal (10)
    8: [], 9: [], 10: []  # Leaf nodes
}

# Set start and goal nodes
start_node = 1
goal_node = 10

# Perform DFS from start to goal
open_list, closed_list = dfs_with_open_closed(graph, start_node, goal_node)

# Show final state of open and closed lists
print("DFS Open List:", open_list)
print("DFS Closed List:", closed_list)
```

Line-by-Line Explanation

Line	Explanation
from collections import deque	Importing deque; used in BFS, not required in DFS.
def bfs_with_open_closed(...)	Defines BFS logic — not used in this activity.
def dfs_with_open_closed(...)	Defines DFS function with tracking of open/closed nodes.
open_list = [start]	Initialize stack with start node.
closed_list = []	Initialize empty list to track visited nodes.
while open_list:	Loop as long as stack isn't empty.
node = open_list.pop()	LIFO: pop last item from the stack.
closed_list.append(node)	Mark the current node as visited.
if node == goal:	Stop if goal is found.
for neighbor in reversed(graph[node]):	Traverse neighbors in reverse to simulate left-to-right DFS.
if neighbor not in open_list...	Avoid re-visiting already seen nodes.
open_list.append(neighbor)	Push unvisited neighbors onto stack.
return open_list, closed_list	Return the lists once search is complete.
graph = {...}	The tree/graph structure in adjacency list form.
start_node = 1	Initial node = 1.
goal_node = 10	Target node = 10.
open_list, closed_list = dfs_with_open_closed(...)	Execute DFS search.
print(...)	Print remaining nodes in stack and visited sequence.

Manual DFS Evaluation (Start = 1, Goal = 10)

Traversal Path:

1 → 2 → 4 (dead end)

Backtrack → 5 → 8 (dead end)

Backtrack → 9 (dead end)
Backtrack → 3 → 6 (dead end)
Backtrack → 7 → 10 (goal found)

Step	Stack (Open List)	Visited (Closed List)
1	[1]	[]
2	[]	[1]
3	[3, 2]	[1]
4	[3]	[1, 2]
5	[3, 5, 4]	[1, 2]
6	[3, 5]	[1, 2, 4]
7	[3, 5, 9, 8]	[1, 2, 4, 5]
8	[3, 5, 9]	[1, 2, 4, 5, 8]
9	[3, 5]	[1, 2, 4, 5, 8, 9]
10	[3]	[1, 2, 4, 5, 8, 9]
11	[7, 6]	[1, 2, 4, 5, 8, 9, 3]
12	[7]	[1, 2, 4, 5, 8, 9, 3, 6]
13	[10]	[1, 2, 4, 5, 8, 9, 3, 6, 7]
14	[]	[1, 2, 4, 5, 8, 9, 3, 6, 7, 10]

Goal (10) Found at Step 14.
Let me know if you also want to draw the graph or compare DFS vs BFS behavior!

```
PS D:\Sir Syed University\6th Semester\AI Lab\Lab py> python labtask.py
DFS Open List: []
DFS Closed List: [1, 2, 4, 5, 8, 9, 3, 6, 7, 10]
```

Activity- 5:
Repeat Activity-4, apply DFS by taking initial and final state as user input. Show results in form of open and closed list. Also evaluate it manually.

✅ **Code with In-Line Comments**

```
from collections import deque  # Import deque (used in BFS, but not DFS here)

# BFS function (not used in Activity 5, included for reference)
def bfs_with_open_closed(graph, start, goal):
    open_list = deque([start])  # Open list using queue for BFS
    closed_list = []  # Visited/Closed list

    while open_list:
        node = open_list.popleft()  # Remove first element from queue (FIFO)
        closed_list.append(node)  # Mark node as visited

        if node == goal:  # Check if goal reached
            return open_list, closed_list

        for neighbor in graph[node]:  # Check neighbors
            if neighbor not in open_list and neighbor not in closed_list:
                open_list.append(neighbor)  # Add unvisited neighbor to open list

    return open_list, closed_list  # If goal not found, return both lists

# DFS function with open and closed list tracking
def dfs_with_open_closed(graph, start, goal):
    open_list = [start]  # Open list as a stack (LIFO)
    closed_list = []  # Closed list for visited nodes

    while open_list:  # While there are nodes to explore
```

```
        node = open_list.pop()  # Pop last node (DFS logic)
        closed_list.append(node)  # Mark node as visited

        if node == goal:  # If goal is found, return the lists
            return open_list, closed_list

        # Add neighbors in reverse order to ensure left-to-right DFS traversal
        for neighbor in reversed(graph[node]):
            if neighbor not in open_list and neighbor not in closed_list:
                open_list.append(neighbor)  # Push unvisited neighbor onto stack

    return open_list, closed_list  # Return final open and closed lists if goal not found

# Graph structure represented as an adjacency list
graph = {
    1: [2, 3],      # Node 1 connects to 2 and 3
    2: [4, 5],      # Node 2 connects to 4 and 5
    3: [6, 7],      # Node 3 connects to 6 and 7
    4: [],          # Node 4 has no children
    5: [8, 9],      # Node 5 connects to 8 and 9
    6: [],          # Leaf node
    7: [10],        # Node 7 connects to goal node 10
    8: [], 9: [], 10: []  # Leaf nodes
}

# Take user input for start and goal nodes
start_node = int(input("Enter the initial state: "))  # Convert input to integer
goal_node = int(input("Enter the goal state: "))      # Convert input to integer

# Call DFS function using user-provided start and goal
open_list, closed_list = dfs_with_open_closed(graph, start_node, goal_node)

# Display results
print("DFS Open List:", open_list)        # Remaining nodes in stack (open list)
print("DFS Closed List:", closed_list)    # Path explored (visited nodes)
```

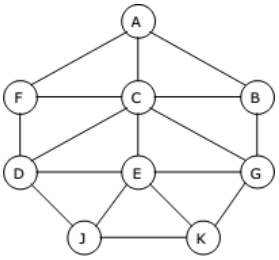
Line-by-Line Explanation

Line	Explanation
from collections import deque	Imports deque, useful for BFS (not used in DFS).
def bfs_with_open_closed...	BFS function – not used here, present just for comparison.
def dfs_with_open_closed...	Defines the DFS algorithm using a stack and visited list.
open_list = [start]	Initialize DFS stack with starting node.
closed_list = []	Create a list to store visited nodes.
while open_list:	Loop until all possible paths are explored or goal is found.
node = open_list.pop()	DFS uses LIFO; pop the last node from stack.
closed_list.append(node)	Mark node as visited.
if node == goal:	Stop search when the goal node is found.
for neighbor in reversed(graph[node])	Add neighbors in reverse for correct left-to-right DFS order.
if neighbor not in open_list...	Avoid cycles by checking both open and closed lists.
open_list.append(neighbor)	Push valid neighbor to the stack.
graph = {...}	Graph represented using adjacency list.
start_node = int(input(...))	Get initial node from user.
goal_node = int(input(...))	Get goal node from user.
dfs_with_open_closed(...)	Perform DFS using user inputs.
print(...)	Output open (remaining) and closed (visited) nodes.

Manual Evaluation Tip (DFS)
If user inputs 1 (start) and 10 (goal):
Closed List (visited path) likely to be:
[1, 2, 4, 5, 8, 9, 3, 6, 7, 10]
Open List (remaining nodes in stack) will be empty when the goal is reached.
Let me know if you'd like a **graph visualization** or **comparison with BFS!**
Or try running it with different start/goal values like 2 → 9 or 3 → 10 to explore.

```
PS D:\Sir Syed University\6th Semester\AI Lab\Lab py> python labtask.py
Enter the initial state: 1
Enter the goal state: 6
DFS Open List: [7]
DFS Closed List: [1, 2, 4, 5, 8, 9, 3, 6]
```

Activity- 6:
Represent a graph with adjacency list using dictionaries. The keys of the dictionary represent nodes; the values have a list of neighbors. Apply Breadth First Search on following graph considering the initial state is A and final state is K. Also evaluate it manually.



Code with Comments

```
from collections import deque  # Import deque to use it as a queue for BFS

def bfs_with_open_closed(graph, start, goal):
    open_list = deque([start])  # Initialize open list (queue) with the start node
    closed_list = []  # Initialize closed list to store visited nodes

    while open_list:  # Continue looping as long as there are nodes to explore
        node = open_list.popleft()  # Dequeue the front node from the open list
        closed_list.append(node)  # Add the node to closed list (visited)

        if node == goal:  # Check if the current node is the goal
            return open_list, closed_list  # If yes, return the current state of both lists

        for neighbor in graph[node]:  # Iterate through neighbors of the current node
            if neighbor not in open_list and neighbor not in closed_list:
                open_list.append(neighbor)  # Add unvisited neighbors to the open list

    return open_list, closed_list  # If goal not found, return open and closed lists

# Define the graph using an adjacency list (dictionary)
graph = {
    'A': ['B', 'C', 'F'],      # A connects to B, C, and F
    'B': ['A', 'G'],           # B connects to A and G
    'C': ['A', 'D', 'E'],      # C connects to A, D, and E
    'D': ['C', 'F', 'J'],      # D connects to C, F, and J
    'E': ['C', 'G', 'J', 'K'],  # E connects to C, G, J, and K
    'F': ['A', 'D'],           # F connects to A and D
    'G': ['B', 'E'],           # G connects to B and E
    'J': ['D', 'E'],           # J connects to D and E
    'K': ['E']                 # K connects only to E
}
```

```
start_node = 'A'  # Set initial node
goal_node = 'K'   # Set goal node

open_list, closed_list = bfs_with_open_closed(graph, start_node, goal_node)  # Call BFS function

print("BFS Open List:", list(open_list))  # Print remaining nodes in queue
print("BFS Closed List:", closed_list)    # Print all visited nodes in order
```

Explanation (Line-by-Line)

Line	Explanation
from collections import deque	deque is used for efficient FIFO operations needed for BFS.
def bfs_with_open_closed(...)	Defines a function to perform BFS and track open/closed lists.
open_list = deque([start])	Starts with the initial node in the queue (open list).
closed_list = []	Keeps a list of all visited (explored) nodes.
while open_list:	Loop continues while nodes are available to explore.
node = open_list.popleft()	Takes the next node to explore from the queue.
closed_list.append(node)	Marks the node as visited.
if node == goal:	If the current node is the target, stop and return.
for neighbor in graph[node]:	Look through all directly connected neighbors.
if neighbor not in ...	Skip neighbors already visited or waiting to be explored.
open_list.append(neighbor)	Add new neighbors to the queue for future exploration.
return open_list, closed_list	Return both lists once BFS completes.
graph = {...}	Represents the graph using a dictionary adjacency list.
start_node = 'A'	Starting node is 'A'.
goal_node = 'K'	Goal node is 'K'.
open_list, closed_list = ...	Calls the function with the graph and input values.
print(...)	Displays final open and closed lists to the user.

Manual BFS Evaluation

Start: A
Goal: K
Step-by-step BFS path (Closed List):
['A', 'B', 'C', 'F', 'G', 'D', 'E', 'J', 'K'] ✅

Let me know if you'd like a **graph diagram**, **DFS version**, or **visual trace table** for this!

```
PS D:\Sir Syed University\6th Semester\AI Lab\Lab py> python labtask.py
BFS Open List: []
BFS Closed List: ['A', 'B', 'C', 'F', 'G', 'D', 'E', 'J', 'K']
```

Activity- 7:

Represent a graph with adjacency list using dictionaries. The keys of the dictionary represent nodes; the values have a list of neighbors. Apply Depth First Search on Activity-6 graph considering the initial state is A and final state is K. Also evaluate it manually.

✅ Code with Comments

```
from collections import deque  # Import deque for BFS queue (even though unused in DFS)

def bfs_with_open_closed(graph, start, goal):
    open_list = deque([start])  # Initialize the open list (queue) with the start node
    closed_list = []  # List to keep track of visited (closed) nodes

    while open_list:  # Continue while there are nodes to explore
        node = open_list.popleft()  # Dequeue node from front (FIFO)
        closed_list.append(node)  # Mark as visited
```

```
        if node == goal:  # Goal found, return current open and closed lists
            return open_list, closed_list

        for neighbor in graph[node]:  # Traverse neighbors
            if neighbor not in open_list and neighbor not in closed_list:
                open_list.append(neighbor)  # Add unvisited neighbors to open list

    return open_list, closed_list  # Return both lists when search ends

def dfs_with_open_closed(graph, start, goal):
    open_list = [start]  # Initialize open list as a stack for DFS (LIFO)
    closed_list = []  # Closed list for visited nodes

    while open_list:  # Loop while nodes remain in stack
        node = open_list.pop()  # Pop the last inserted node (DFS behavior)
        closed_list.append(node)  # Mark the node as visited

        if node == goal:  # If goal is reached, return lists
            return open_list, closed_list

        # Reverse the neighbors to maintain correct DFS order (left to right)
        for neighbor in reversed(graph[node]):
            if neighbor not in open_list and neighbor not in closed_list:
                open_list.append(neighbor)  # Push unvisited neighbors

    return open_list, closed_list  # Return lists if goal not found

# Define the graph using adjacency list (dictionary)
graph = {
    'A': ['B', 'C', 'F'],
    'B': ['A', 'G'],
    'C': ['A', 'D', 'E'],
    'D': ['C', 'F', 'J'],
    'E': ['C', 'G', 'J', 'K'],
    'F': ['A', 'D'],
    'G': ['B', 'E'],
    'J': ['D', 'E'],
    'K': ['E']
}

start_node = 'A'  # Initial state set to 'A'
goal_node = 'K'   # Final goal state is 'K'

# Perform BFS (optional step, as required by code)
open_list, closed_list = bfs_with_open_closed(graph, start_node, goal_node)
print("BFS Open List:", list(open_list))  # Display remaining nodes in BFS open list
print("BFS Closed List:", closed_list)    # Display BFS visited nodes

# Perform DFS
open_list, closed_list = dfs_with_open_closed(graph, start_node, goal_node)
print("DFS Open List:", open_list)        # Display remaining nodes in DFS open list
print("DFS Closed List:", closed_list)    # Display DFS visited nodes
```

Explanation (Line-by-Line)

Line	Explanation
from collections import deque	Imports deque for BFS queue behavior (FIFO).

def bfs_with_open_closed(...)	BFS function using open/closed list mechanism.
open_list = deque([start])	Starts BFS with the initial node in the queue.
closed_list = []	Tracks visited nodes to avoid re-visits.
while open_list:	Loop continues until all paths are explored.
node = open_list.popleft()	Dequeues front node (BFS behavior).
if node == goal:	If goal is found, return open/closed lists.
for neighbor in graph[node]:	Explore each neighbor of current node.
if neighbor not in ...	Avoid re-exploration.
def dfs_with_open_closed(...)	DFS function (depth-first logic).
open_list = [start]	DFS starts with the stack containing the start node.
node = open_list.pop()	Pops last element (LIFO).
for neighbor in reversed(...)	Neighbors reversed to maintain DFS ordering.
graph = {...}	Graph represented as an adjacency list.
start_node = 'A'	Starting node.
goal_node = 'K'	Target node.
open_list, closed_list = ...	Run BFS, then DFS.
print(...)	Output open and closed lists for BFS and DFS.

Manual DFS Evaluation

Initial: A

Goal: K

Traversal Path (Closed List):

A → F → D → J → E → K

So the **DFS Closed List =** ['A', 'F', 'D', 'J', 'E', 'K'] ✅

The DFS stops at K once reached.

```
PS D:\Sir Syed University\6th Semester\AI Lab\Lab py> python labtask.py
BFS Open List: []
BFS Closed List: ['A', 'B', 'C', 'F', 'G', 'D', 'E', 'J', 'K']
DFS Open List: ['F', 'C']
DFS Closed List: ['A', 'B', 'G', 'E', 'J', 'D', 'K']
```

Activity 08:

You need to implement DFS and BFS both using "Pygame" and "Pyamaze" library, generate the maze of (20,20) and find the path.

✅ **Code with Inline Comments**

```
from pyamaze import maze, agent, textLabel  # Import required classes from pyamaze

def BFS(m):  # Define BFS function with maze object as input
    start = (m.rows, m.cols)  # Starting cell is the bottom-right corner of the maze
    frontier = [start]  # Frontier list initialized with the start cell (acts like a queue)
    explored = [start]  # Explored list to keep track of visited cells
    bfsPath = {}  # Dictionary to store path history (child: parent)

    while len(frontier) > 0:  # Loop until there are no more nodes in the frontier
        currCell = frontier.pop(0)  # Dequeue the first cell in the frontier (BFS behavior)
        if currCell == (1, 1):  # If goal (top-left) is reached, stop
            break

        for d in 'ESNW':  # Explore all directions: East, South, North, West
            if m.maze_map[currCell][d] == True:  # If path exists in that direction
                # Determine the neighboring cell based on direction
                if d == 'E':
                    childCell = (currCell[0], currCell[1] + 1)
                elif d == 'W':
                    childCell = (currCell[0], currCell[1] - 1)
```

```
            elif d == 'N':
                childCell = (currCell[0] - 1, currCell[1])
            elif d == 'S':
                childCell = (currCell[0] + 1, currCell[1])

            if childCell in explored:  # Skip already visited cells
                continue

            frontier.append(childCell)  # Add to frontier
            explored.append(childCell)  # Mark as explored
            bfsPath[childCell] = currCell  # Store the parent of child cell

    fwdPath = {}  # Dictionary to store the final path from start to goal
    cell = (1, 1)  # Start from the goal cell
    while cell != start:  # Reconstruct the path by going backwards
        fwdPath[bfsPath[cell]] = cell
        cell = bfsPath[cell]

    return fwdPath  # Return the full path

if __name__ == '__main__':  # Main program starts here
    m = maze(10, 10)  # Create a 10x10 maze object
    m.CreateMaze(loopPercent=100)  # Generate the maze with 100% loop (more complex)

    path = BFS(m)  # Call BFS function to find the path

    a = agent(m, footprints=True)  # Create an agent that leaves footprints
    m.tracePath({a: path})  # Trace the path found by BFS on the maze using the agent

    l = textLabel(m, 'Length of Shortest Path', len(path) + 1)  # Label to show path length
    print('Length of Shortest Path', len(path) + 1)  # Print the length of the path
    print(m.maze_map)  # Print the maze structure (for debug/reference)

    m.run()  # Run the Pygame maze window
```

Explanation (Line-by-Line)

Line	Explanation
from pyamaze import ...	Imports the maze-building and visualization tools.
def BFS(m):	Defines the BFS search function for a given maze m.
start = (m.rows, m.cols)	Sets the starting point as the bottom-right of the maze.
frontier = [start]	BFS queue initialized with the start cell.
explored = [start]	Keeps track of all visited nodes to avoid reprocessing.
bfsPath = {}	Stores the parent of each cell for path reconstruction.
while len(frontier) > 0:	Continue until there are no nodes left to explore.
currCell = frontier.pop(0)	Gets the first node in the frontier (FIFO order).
if currCell == (1, 1): break	Stop the search if goal (top-left) is found.
for d in 'ESNW':	Loop over all possible directions.
if m.maze_map[currCell][d]==True:	Check if that direction is not blocked.
childCell = (...)	Calculate the neighbor cell based on direction.
if childCell in explored: continue	Skip visited cells.
frontier.append(childCell)	Add new cell to the frontier for exploration.
explored.append(childCell)	Mark cell as explored.
bfsPath[childCell] = currCell	Store how the cell was reached (parent link).
fwdPath = {}	Will store the reconstructed shortest path.
cell = (1, 1)	Start from the goal.
while cell != start:	Backtrack to the start node.

fwdPath[bfsPath[cell]] = cell	Reconstruct the path from goal to start.
return fwdPath	Return the final path as a dictionary.
if __name__ == '__main__':	Ensure code runs only when file is run directly.
m = maze(10, 10)	Initialize a 10x10 maze.
m.CreateMaze(loopPercent=100)	Generate the maze with full loop complexity.
path = BFS(m)	Run BFS and get the solution path.
a = agent(m, footprints=True)	Create a visible agent to follow the path.
m.tracePath({a: path})	Trace the BFS path on the maze.
textLabel(...)	Display the path length in the GUI.
print(...)	Debug prints.
m.run()	Launch the Pygame-based maze window.

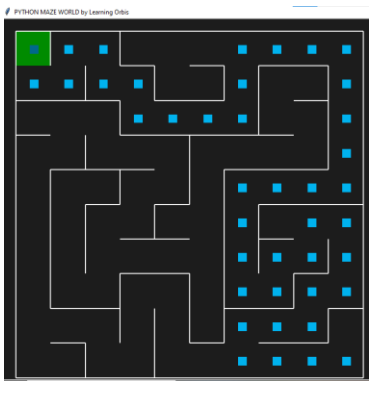
DFS implementation using Pyamaze with:

```
from pyamaze import maze, agent  # Importing maze and agent classes from pyamaze library

def DFS(m):  # Define the DFS function with a maze object as input
    start = (m.rows, m.cols)  # Start cell is set to the bottom-right of the maze
    explored = [start]  # List to store explored (visited) cells
    frontier = [start]  # Stack used for DFS (LIFO behavior)
    dfsPath = {}  # Dictionary to store the DFS path (child: parent)

    while len(frontier) > 0:  # Continue while there are unexplored cells
        currCell = frontier.pop()  # Get the last added cell (DFS: LIFO)
        if currCell == (1, 1):  # If goal (top-left cell) is reached, stop
            break

        for d in 'ESNW':  # Loop through all directions: East, South, North, West
            if m.maze_map[currCell][d] == True:  # Check if there is a path in this direction
                # Determine the child cell based on direction
                if d == 'E':
                    childCell = (currCell[0], currCell[1] + 1)
                elif d == 'W':
                    childCell = (currCell[0], currCell[1] - 1)
                elif d == 'S':
                    childCell = (currCell[0] + 1, currCell[1])
                elif d == 'N':
                    childCell = (currCell[0] - 1, currCell[1])

                if childCell in explored:  # Skip if already explored
                    continue

                explored.append(childCell)  # Mark as explored
                frontier.append(childCell)  # Add to stack for further exploration
                dfsPath[childCell] = currCell  # Store parent for path reconstruction

    fwdPath = {}  # Dictionary to hold the final forward path from start to goal
    cell = (1, 1)  # Begin from the goal cell
    while cell != start:  # Backtrack from goal to start
        fwdPath[dfsPath[cell]] = cell  # Rebuild path using parent references
        cell = dfsPath[cell]

    return fwdPath  # Return the complete forward path

if __name__ == '__main__':  # Ensures this code runs only if the file is executed directly
    m = maze(10, 10)  # Create a 10x10 maze object
```

```
m.CreateMaze()  # Generate the maze layout

path = DFS(m)  # Call DFS and get the path

a = agent(m, footprints=True)  # Create an agent that will follow the path, showing footprints
m.tracePath({a: path})  # Trace the path in the maze using the agent

# print({a:path})  # (Optional) You can print the path dictionary for debugging

m.run()  # Run the maze game window (visualization)
```

Line-by-Line Explanation

Line	Explanation
from pyamaze import maze, agent	Import necessary classes from the pyamaze library.
def DFS(m):	Define a DFS function that accepts a maze object m.
start = (m.rows, m.cols)	Set the starting cell as the bottom-right corner.
explored = [start]	Track all visited cells. Prevents revisiting.
frontier = [start]	Stack for DFS; last-in, first-out behavior.
dfsPath = {}	Dictionary to keep track of parent relationships.
while len(frontier) > 0:	Continue until all reachable cells are visited.
currCell = frontier.pop()	DFS uses the last element (stack behavior).
if currCell == (1, 1): break	Stop if goal (top-left) is reached.
for d in 'ESNW':	Loop over each direction.
if m.maze_map[currCell][d]==True:	Check if there's a path in that direction.
childCell = ...	Determine the neighboring cell.
if childCell in explored: continue	Skip already explored cells.
explored.append(childCell)	Mark child as explored.
frontier.append(childCell)	Add child to the frontier for later exploration.
dfsPath[childCell] = currCell	Record the parent of the child cell.
fwdPath = {}	Dictionary to reconstruct the actual path from goal to start.
cell = (1, 1)	Start from the goal.
while cell != start:	Continue until we reach the start cell.
fwdPath[dfsPath[cell]] = cell	Rebuild the forward path from DFS record.
cell = dfsPath[cell]	Move backward to the parent cell.
return fwdPath	Return the full path from start to goal.
if __name__ == '__main__':	Main block to run this code.
m = maze(10, 10)	Create a 10×10 maze object.
m.CreateMaze()	Generate the maze with random paths.
path = DFS(m)	Call DFS and store the resulting path.
a = agent(m, footprints=True)	Create an agent that follows and shows path.
m.tracePath({a: path})	Draw the path for the agent in the maze.
m.run()	Launch the maze window (Pygame-based).



Here's the **DFS Maze Solver using Pygame** with:

✅ **Inline comments on every line**
✅ **Detailed explanation after the code**

✅ **Code with Inline Comments**

```
import pygame  # Importing the Pygame library
import time    # Importing time module (optional, not used here)

# Initialize Pygame library
pygame.init()

# Set screen dimensions
SCREEN_WIDTH = 800
SCREEN_HEIGHT = 600
CELL_SIZE = 40  # Each maze cell will be 40x40 pixels

# Define some RGB colors
WHITE = (255, 255, 255)  # Open path color
BLACK = (0, 0, 0)        # Wall color
RED = (255, 0, 0)        # Player color
BLUE = (0, 0, 255)       # Path color
GREEN = (0, 255, 0)      # Not used, but reserved

# Create the Pygame window
screen = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))
pygame.display.set_caption("DFS Visualization")  # Set window title

# Maze dimensions (10x10)
ROWS = 10
COLS = 10

# Maze layout (1=open, 0=wall)
maze = [
    [1, 1, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, 1, 1, 1, 0, 0, 0, 0, 0, 0],
    [0, 0, 0, 1, 0, 0, 0, 0, 0, 0],
    [0, 0, 0, 1, 1, 1, 1, 1, 0, 0],
    [0, 0, 0, 0, 0, 0, 0, 1, 0, 0],
    [0, 0, 0, 0, 0, 0, 0, 1, 1, 1],
    [0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
    [0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
    [0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
    [0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
]

# DFS algorithm function
def dfs(maze, start, goal):
    stack = [start]        # Stack to hold next nodes to explore
    visited = set()        # Track visited cells
    path = {}              # Store child:parent relationship

    while stack:
```

```
        current = stack.pop()  # LIFO: pop last inserted cell
        if current == goal:    # Goal found
            break
        visited.add(current)  # Mark cell as visited

        # Check all 4 directions (Right, Down, Left, Up)
        for direction in [(0, 1), (1, 0), (0, -1), (-1, 0)]:
            next_cell = (current[0] + direction[0], current[1] + direction[1])
            # Check bounds and if next cell is open and not visited
            if (
                0 <= next_cell[0] < ROWS and
                0 <= next_cell[1] < COLS and
                next_cell not in visited and
                maze[next_cell[0]][next_cell[1]] == 1
            ):
                stack.append(next_cell)       # Add to stack
                path[next_cell] = current     # Track the path

    # Reconstruct full path from goal to start
    full_path = []
    cell = goal
    while cell != start:
        full_path.append(cell)
        cell = path[cell]
    full_path.append(start)
    full_path.reverse()  # Start to goal
    return full_path

# Draw maze and path
def draw_maze(maze, path, player_pos):
    for row in range(ROWS):
        for col in range(COLS):
            color = WHITE if maze[row][col] == 1 else BLACK  # Open or wall
            pygame.draw.rect(screen, color, (col * CELL_SIZE, row * CELL_SIZE, CELL_SIZE, CELL_SIZE))
            pygame.draw.rect(screen, BLACK, (col * CELL_SIZE, row * CELL_SIZE, CELL_SIZE, CELL_SIZE), 1)  # Grid border

    # Draw the path in BLUE
    for cell in path:
        pygame.draw.rect(screen, BLUE, (cell[1] * CELL_SIZE, cell[0] * CELL_SIZE, CELL_SIZE, CELL_SIZE))

    # Draw player in RED
    pygame.draw.rect(screen, RED, (player_pos[1] * CELL_SIZE, player_pos[0] * CELL_SIZE, CELL_SIZE, CELL_SIZE))

# Main game loop
def main():
    start = (0, 0)  # Start position (top-left)
    goal = (9, 9)   # Goal position (bottom-right)
    path = dfs(maze, start, goal)  # Get DFS path
    player_pos = start           # Player starts at start

    run = True
    clock = pygame.time.Clock()  # Control frame rate

    while run:
        screen.fill(BLACK)              # Clear screen
        draw_maze(maze, path, player_pos) # Redraw maze
```

```
    for event in pygame.event.get():
        if event.type == pygame.QUIT: # Exit window
            run = False
        elif event.type == pygame.KEYDOWN:  # Move on key press
            if event.key == pygame.K_w:  # Up
                next_pos = (player_pos[0] - 1, player_pos[1])
            elif event.key == pygame.K_s:  # Down
                next_pos = (player_pos[0] + 1, player_pos[1])
            elif event.key == pygame.K_a:  # Left
                next_pos = (player_pos[0], player_pos[1] - 1)
            elif event.key == pygame.K_d:  # Right
                next_pos = (player_pos[0], player_pos[1] + 1)
            else:
                next_pos = player_pos  # Invalid key, no move

            # If move is within bounds and not a wall
            if (
                0 <= next_pos[0] < ROWS and
                0 <= next_pos[1] < COLS and
                maze[next_pos[0]][next_pos[1]] == 1
            ):
                player_pos = next_pos  # Move player

    pygame.display.update()  # Refresh screen
    clock.tick(60)          # Limit to 60 frames/sec

  pygame.quit()  # Exit game

# Run the program
if __name__ == "__main__":
  main()
```

Line-by-Line Explanation

Part	Explanation
pygame.init()	Starts all the Pygame modules.
screen = pygame.display.set_mode(...)	Creates a window for drawing.
maze	2D list to define which cells are open or walls.
dfs(...)	DFS is implemented using a stack to explore paths.
stack.pop()	DFS pops from the end (last-in).
visited	Keeps track of cells that have been explored.
path	Stores child:parent mapping to reconstruct the path later.
draw_maze(...)	Visualizes the maze and colors the solution path and player.
main()	Core game loop, handles drawing and player movement.
pygame.event.get()	Listens for quit or key press events.
pygame.quit()	Closes the window and stops the program.

Code with Comments:

```
import pygame  # Import pygame for visualization
from collections import deque  # Import deque for queue (used in BFS)

# Initialize Pygame
pygame.init()
```

```
# Screen dimensions
SCREEN_WIDTH = 800
SCREEN_HEIGHT = 600
CELL_SIZE = 40  # Size of each cell in the grid

# Define RGB color values
WHITE = (255, 255, 255)  # Open path color
BLACK = (0, 0, 0)        # Wall and background
RED = (255, 0, 0)        # Player position
BLUE = (0, 0, 255)       # Path color
GREEN = (0, 255, 0)      # Unused here, but can be used for visited cells

# Create the display screen
screen = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))
pygame.display.set_caption("BFS Visualization")  # Set the title

# Maze grid dimensions
ROWS = 10
COLS = 10

# Maze layout: 1 = path, 0 = wall
maze = [
    [1, 1, 0, 0, 0, 0, 0, 0, 0, 0],
    [0, 1, 1, 1, 0, 0, 0, 0, 0, 0],
    [0, 0, 0, 1, 0, 0, 0, 0, 0, 0],
    [0, 0, 0, 1, 1, 1, 1, 1, 0, 0],
    [0, 0, 0, 0, 0, 0, 0, 1, 0, 0],
    [0, 0, 0, 0, 0, 0, 0, 1, 1, 1],
    [0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
    [0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
    [0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
    [0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
]

# Breadth-First Search (BFS) function
def bfs(maze, start, goal):
    queue = deque([start])  # Initialize queue with start node
    visited = set()  # Track visited cells
    visited.add(start)
    path = {}  # Store parent of each visited node

    while queue:
        current = queue.popleft()  # Dequeue front element
        if current == goal:  # Goal found
            break

        # Explore neighbors: right, down, left, up
        for direction in [(0, 1), (1, 0), (0, -1), (-1, 0)]:
            next_cell = (current[0] + direction[0], current[1] + direction[1])
            if (
                0 <= next_cell[0] < ROWS and
                0 <= next_cell[1] < COLS and
                next_cell not in visited and
                maze[next_cell[0]][next_cell[1]] == 1
            ):
                queue.append(next_cell)
                visited.add(next_cell)
```

```
            path[next_cell] = current  # Record path

    # Reconstruct path from goal to start
    full_path = []
    cell = goal
    while cell != start:
        full_path.append(cell)
        cell = path[cell]
    full_path.append(start)
    full_path.reverse()  # Reverse to get path from start to goal
    return full_path

# Function to draw the maze, path, and player
def draw_maze(maze, path, player_pos):
    for row in range(ROWS):
        for col in range(COLS):
            color = WHITE if maze[row][col] == 1 else BLACK  # White = open, Black = wall
            pygame.draw.rect(screen, color, (col * CELL_SIZE, row * CELL_SIZE, CELL_SIZE, CELL_SIZE))
            pygame.draw.rect(screen, BLACK, (col * CELL_SIZE, row * CELL_SIZE, CELL_SIZE, CELL_SIZE), 1)  # Border

    # Draw the BFS path in blue
    for cell in path:
        pygame.draw.rect(screen, BLUE, (cell[1] * CELL_SIZE, cell[0] * CELL_SIZE, CELL_SIZE, CELL_SIZE))

    # Draw the player in red
    pygame.draw.rect(screen, RED, (player_pos[1] * CELL_SIZE, player_pos[0] * CELL_SIZE, CELL_SIZE, CELL_SIZE))

# Main game loop
def main():
    start = (0, 0)  # Starting cell
    goal = (9, 9)  # Goal cell
    path = bfs(maze, start, goal)  # Run BFS and get path
    player_pos = start  # Player starts at the beginning

    run = True
    clock = pygame.time.Clock()  # Control frame rate

    while run:
        screen.fill(BLACK)  # Clear screen
        draw_maze(maze, path, player_pos)  # Draw maze and path

        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                run = False
            elif event.type == pygame.KEYDOWN:
                if event.key == pygame.K_w:  # Up
                    next_pos = (player_pos[0] - 1, player_pos[1])
                elif event.key == pygame.K_s:  # Down
                    next_pos = (player_pos[0] + 1, player_pos[1])
                elif event.key == pygame.K_a:  # Left
                    next_pos = (player_pos[0], player_pos[1] - 1)
                elif event.key == pygame.K_d:  # Right
                    next_pos = (player_pos[0], player_pos[1] + 1)
                else:
                    next_pos = player_pos

                # Validate move
```

```
            if (
                0 <= next_pos[0] < ROWS and
                0 <= next_pos[1] < COLS and
                maze[next_pos[0]][next_pos[1]] == 1
            ):
                player_pos = next_pos  # Move player

        pygame.display.update()  # Refresh screen
        clock.tick(60)  # Limit to 60 FPS

    pygame.quit()  # Quit game

# Start the game
if __name__ == "__main__":
    main()
```

✅ **Line-by-Line Explanation:**

1. import pygame and from collections import deque: Import Pygame and deque for BFS.
2. pygame.init(): Initializes all Pygame modules.
3. Set screen size and cell size.
4. Define colors using RGB tuples.
5. pygame.display.set_mode: Creates a window.
6. maze: A 10x10 matrix representing walls (0) and paths (1).
7. bfs(): Implements BFS with a queue, tracks visited nodes, and builds a path dictionary.
8. Path is reconstructed from goal back to start using path dictionary.
9. draw_maze(): Draws the maze grid, path (in blue), and player (in red).
10. main(): Contains game loop:
    - Calls BFS to get path.
    - Handles key inputs (WASD).
    - Updates player position and redraws screen.
11. pygame.quit(): Cleans up resources when the game window is closed.

Let me know if you want to visualize BFS step-by-step or compare BFS with DFS side-by-side!

Here's the full code for **both Activity 1 and Activity 2** with **comments next to each line**, followed by a **clear explanation of each line/block** of code.

✅ **Activity 1: BFS, DFS, and A* Search**

Code with Comments:

```
from collections import deque  # For BFS queue
import heapq  # For A* priority queue

# BFS Function
def bfs_with_open_closed(graph, start, goal):
    open_list = deque([start])  # FIFO queue for BFS
    closed_list = []  # Stores visited nodes

    while open_list:
        node = open_list.popleft()  # Get next node
        closed_list.append(node)  # Mark it visited

        if node == goal:  # If goal is found
            return open_list, closed_list

        for neighbor in graph[node]:  # Check neighbors
```

```
            if neighbor not in open_list and neighbor not in closed_list:
                open_list.append(neighbor)  # Add unvisited neighbors

    return open_list, closed_list  # If goal not found

# DFS Function
def dfs_with_open_closed(graph, start, goal):
    open_list = [start]  # LIFO stack for DFS
    closed_list = []

    while open_list:
        node = open_list.pop()  # Get top node
        closed_list.append(node)

        if node == goal:
            return open_list, closed_list

        for neighbor in reversed(graph[node]):  # Reverse for DFS order
            if neighbor not in open_list and neighbor not in closed_list:
                open_list.append(neighbor)

    return open_list, closed_list

# A* Search
def a_star_search(graph, heuristic, start, goal):
    open_list = []  # Priority queue (min-heap)
    heapq.heappush(open_list, (0 + heuristic[start], start))  # Push (f, node)
    closed_list = []
    g_costs = {start: 0}  # Cost so far
    came_from = {}  # Path reconstruction

    while open_list:
        _, current = heapq.heappop(open_list)  # Get node with lowest f
        closed_list.append(current)

        if current == goal:
            path = reconstruct_path(came_from, current)
            return path, closed_list

        for neighbor, cost in graph[current]:
            tentative_g_cost = g_costs[current] + cost
            if neighbor not in g_costs or tentative_g_cost < g_costs[neighbor]:
                g_costs[neighbor] = tentative_g_cost
                f_cost = tentative_g_cost + heuristic[neighbor]
                heapq.heappush(open_list, (f_cost, neighbor))
                came_from[neighbor] = current

    return None, closed_list

# Reconstruct shortest path from came_from dictionary
def reconstruct_path(came_from, current):
    path = [current]
    while current in came_from:
        current = came_from[current]
        path.append(current)
    path.reverse()
    return path
```

```
# Graph for BFS/DFS
graph_bfs_dfs = {
    'A': ['B', 'C', 'F'],
    'B': ['A', 'G'],
    'C': ['A', 'D', 'E'],
    'D': ['C', 'F', 'J'],
    'E': ['C', 'G', 'J', 'K'],
    'F': ['A', 'D'],
    'G': ['B', 'E'],
    'J': ['D', 'E'],
    'K': ['E']
}

# A* Graph (with weights)
graph_a_star = {
    'S': [('A', 2), ('B', 2)],
    'A': [('G', 2)],
    'B': [('G', 3)],
    'G': []
}

# Heuristics
heuristic = {
    'S': 3,
    'A': 2,
    'B': 1,
    'G': 0
}

# Run BFS
start_node_bfs_dfs = 'A'
goal_node_bfs_dfs = 'K'
open_list, closed_list = bfs_with_open_closed(graph_bfs_dfs, start_node_bfs_dfs, goal_node_bfs_dfs)
print("BFS Open List:", list(open_list))
print("BFS Closed List:", closed_list)

# Run DFS
open_list, closed_list = dfs_with_open_closed(graph_bfs_dfs, start_node_bfs_dfs, goal_node_bfs_dfs)
print("DFS Open List:", open_list)
print("DFS Closed List:", closed_list)

# Run A*
start_node_a_star = 'S'
goal_node_a_star = 'G'
path, closed_list = a_star_search(graph_a_star, heuristic, start_node_a_star, goal_node_a_star)
print("Shortest Path:", path)
print("Closed List:", closed_list)
```

Line-by-Line Explanation:

- bfs_with_open_closed: Implements BFS with deque for queue and a closed_list to track visited nodes.
- dfs_with_open_closed: Implements DFS with stack (list), reversing neighbors to preserve DFS order.
- a_star_search: Implements A* with a min-heap (heapq), maintaining g_cost (cost-so-far) and using heuristic.
- reconstruct_path: Builds the path from goal to start using backtracking.
- Graphs are defined manually based on nodes and their connections or edge weights.
- Heuristic values estimate distance to the goal.

- Finally, each algorithm is called, and the result is printed.

✅ **Activity 2: A* from s to c or d**

Code with Comments:

```
import heapq  # For priority queue

# A* algorithm for multiple goal nodes
def a_star_search(graph, heuristic, start, goals):
    open_list = []  # Priority queue
    heapq.heappush(open_list, (0 + heuristic[start], start))  # Push start node
    closed_list = []
    g_costs = {start: 0}
    came_from = {}

    while open_list:
        _, current = heapq.heappop(open_list)  # Node with smallest f(n)
        closed_list.append(current)

        if current in goals:  # Stop if any goal is found
            path = reconstruct_path(came_from, current)
            return path, closed_list

        for neighbor, cost in graph[current]:
            tentative_g_cost = g_costs[current] + cost
            if neighbor not in g_costs or tentative_g_cost < g_costs[neighbor]:
                g_costs[neighbor] = tentative_g_cost
                f_cost = tentative_g_cost + heuristic[neighbor]
                heapq.heappush(open_list, (f_cost, neighbor))
                came_from[neighbor] = current

    return None, closed_list

# Builds final path
def reconstruct_path(came_from, current):
    path = [current]
    while current in came_from:
        current = came_from[current]
        path.append(current)
    path.reverse()
    return path

# Graph with weighted edges
graph = {
    's': [('a', 3), ('b', 2)],
    'a': [('c', 3), ('d', 5)],
    'b': [('a', 3), ('d', 3)],
    'c': [],
    'd': []
}

# Heuristic for each node
heuristic = {
    's': 1,
    'a': 3,
    'b': 3,
    'c': 0,
```

```
    'd': 0
}

# Start and goal
start_node = 's'
goal_nodes = ['c', 'd']  # Multiple end nodes

# Run A*
path, closed_list = a_star_search(graph, heuristic, start_node, goal_nodes)
print("Shortest Path:", path)
print("Closed List:", closed_list)
```

Line-by-Line Explanation:

- This version of A* allows multiple goals.
- graph: edges with weights.
- heuristic: estimates from each node to nearest goal.
- open_list: stores (f, node) with lowest total cost first.
- closed_list: keeps visited nodes.
- Algorithm stops when any goal node is reached.
- reconstruct_path: traces shortest path.

Let me know if you'd like me to **show manual solutions** for either graph step-by-step or help **visualize them**!

 # --- Activity 3: A* Search from node 'a' to 'z' ---

```
import heapq  # Import heapq for priority queue functionality

# A* Search Function
def a_star_search(graph, heuristic, start, goal):
    open_list = []  # Priority queue to select lowest f(n)
    heapq.heappush(open_list, (0 + heuristic[start], start))  # Push start node with f(n) = g(n) + h(n) = 0 + h(start)
    closed_list = []  # Track visited nodes
    g_costs = {start: 0}  # Cost from start node to current node
    came_from = {}  # To reconstruct path

    while open_list:
        _, current = heapq.heappop(open_list)  # Get node with lowest f(n)
        closed_list.append(current)  # Mark as visited

        if current == goal:  # If goal is reached
            path = reconstruct_path(came_from, current)  # Reconstruct path from goal to start
            return path, closed_list  # Return result

        for neighbor, cost in graph[current]:  # Explore neighbors
            tentative_g_cost = g_costs[current] + cost  # Calculate cost to neighbor
            if neighbor not in g_costs or tentative_g_cost < g_costs[neighbor]:  # If better path found
                g_costs[neighbor] = tentative_g_cost  # Update g-cost
                f_cost = tentative_g_cost + heuristic[neighbor]  # f(n) = g(n) + h(n)
                heapq.heappush(open_list, (f_cost, neighbor))  # Add to open list
                came_from[neighbor] = current  # Track path

    return None, closed_list  # No path found

# Function to reconstruct path from came_from dict
```

```
def reconstruct_path(came_from, current):
    path = [current]  # Start from goal
    while current in came_from:  # Trace back to start
        current = came_from[current]
        path.append(current)
    path.reverse()  # Reverse to get path from start to goal
    return path

# Graph definition
graph = {
    'a': [('b', 9), ('c', 4), ('d', 7)],
    'b': [('e', 11)],
    'c': [('b', 17), ('f', 12)],
    'd': [('c', 14), ('f', 14)],
    'e': [('z', 5)],
    'f': [('e', 8), ('z', 9)],
    'z': []
}

# Heuristic estimates to goal 'z'
heuristic = {
    'a': 21,
    'b': 14,
    'c': 18,
    'd': 18,
    'e': 5,
    'f': 8,
    'z': 0
}

# Set start and goal
start_node = 'a'
goal_node = 'z'

# Run A* Search
path, closed_list = a_star_search(graph, heuristic, start_node, goal_node)

# Show results
print("Shortest Path:", path)
print("Closed List:", closed_list)


# --- Activity 4: A* Search from Arad to Bucharest ---
# Same a_star_search and reconstruct_path function reused

# Graph of Romania map
romania_graph = {
    'Arad': [('Zerind', 75), ('Sibiu', 140), ('Timisoara', 118)],
    'Zerind': [('Arad', 75), ('Oradea', 71)],
    'Oradea': [('Zerind', 71), ('Sibiu', 151)],
    'Sibiu': [('Arad', 140), ('Oradea', 151), ('Fagaras', 99), ('Rimnicu Vilcea', 80)],
    'Timisoara': [('Arad', 118), ('Lugoj', 111)],
    'Lugoj': [('Timisoara', 111), ('Mehadia', 70)],
    'Mehadia': [('Lugoj', 70), ('Drobeta', 75)],
    'Drobeta': [('Mehadia', 75), ('Craiova', 120)],
    'Craiova': [('Drobeta', 120), ('Rimnicu Vilcea', 146), ('Pitesti', 138)],
    'Rimnicu Vilcea': [('Sibiu', 80), ('Craiova', 146), ('Pitesti', 97)],
```

```
    'Fagaras': [('Sibiu', 99), ('Bucharest', 211)],
    'Pitesti': [('Rimnicu Vilcea', 97), ('Craiova', 138), ('Bucharest', 101)],
    'Bucharest': [('Fagaras', 211), ('Pitesti', 101), ('Giurgiu', 90), ('Urziceni', 85)],
    'Giurgiu': [('Bucharest', 90)],
    'Urziceni': [('Bucharest', 85), ('Hirsova', 98), ('Vaslui', 142)],
    'Hirsova': [('Urziceni', 98), ('Eforie', 86)],
    'Eforie': [('Hirsova', 86)],
    'Vaslui': [('Urziceni', 142), ('Iasi', 92)],
    'Iasi': [('Vaslui', 92), ('Neamt', 87)],
    'Neamt': [('Iasi', 87)]
}

# Heuristic values (straight-line distance to Bucharest)
romania_heuristic = {
    'Arad': 366, 'Zerind': 374, 'Oradea': 380, 'Sibiu': 253, 'Timisoara': 329,
    'Lugoj': 244, 'Mehadia': 241, 'Drobeta': 242, 'Craiova': 160, 'Rimnicu Vilcea': 193,
    'Fagaras': 176, 'Pitesti': 100, 'Bucharest': 0, 'Giurgiu': 77, 'Urziceni': 80,
    'Hirsova': 151, 'Eforie': 161, 'Vaslui': 199, 'Iasi': 226, 'Neamt': 234
}

# Run A* search
path, closed_list = a_star_search(romania_graph, romania_heuristic, 'Arad', 'Bucharest')
print("\nShortest Path from Arad to Bucharest:", path)
print("Closed List:", closed_list)


# --- Activity 5: Greedy Best-First Search ---

def greedy_best_first_search(graph, heuristic, start, goal):
    open_list = []  # Priority queue ordered by heuristic
    heapq.heappush(open_list, (heuristic[start], start))  # Push start node with h(n)
    closed_list = []  # List of visited nodes
    came_from = {}  # Track visited path

    while open_list:
        _, current = heapq.heappop(open_list)  # Pick node with lowest h(n)
        closed_list.append(current)  # Mark visited

        if current == goal:
            path = reconstruct_path(came_from, current)  # Return path to goal
            return path, closed_list

        for neighbor, _ in graph[current]:  # Check neighbors
            if neighbor not in closed_list:  # Avoid revisiting
                heapq.heappush(open_list, (heuristic[neighbor], neighbor))  # Push neighbor
                came_from[neighbor] = current  # Track path

    return None, closed_list

# Graph and heuristic for Greedy BFS
graph = {
    'A': [('B', 6), ('D', 6)],
    'B': [('A', 6), ('C', 5), ('E', 10)],
    'C': [('B', 5), ('D', 13), ('F', 7)],
    'D': [('A', 6), ('C', 13), ('H', 4)],
    'E': [('B', 10), ('F', 6)],
    'F': [('C', 7), ('E', 6), ('H', 4)],
```

```
    'H': [('D', 4), ('F', 6), ('G', 3)],
    'G': [('H', 3)]
}

heuristic = {
    'A': 12, 'B': 19, 'C': 13, 'D': 6, 'E': 10,
    'F': 7, 'H': 3, 'G': 0
}

# Run Greedy BFS
path, closed_list = greedy_best_first_search(graph, heuristic, 'A', 'G')
print("\nGreedy Best-First Path:", path)
print("Closed List:", closed_list)
```

Activity- 6:
Implement Best First Search in the following state graph, find the shortest path between node 'S' to Node 'G'. Also solve it manually.
Here is your **Best-First Search implementation** with **comments beside each line** followed by a **detailed explanation** of each part of the code.

✅ Code with Inline Comments

```
import heapq  # Import heapq to implement priority queue (min-heap)

# Function for Best-First Search
def best_first_search(graph, heuristic, start, goal):
    open_list = []  # Priority queue for nodes to be explored (min-heap based on h(n))
    heapq.heappush(open_list, (heuristic[start], start))  # Push the start node with its heuristic value
    closed_list = []  # List to keep track of visited nodes
    came_from = {}  # Dictionary to store the path (where each node came from)

    while open_list:  # Continue until there are no more nodes to explore
        _, current = heapq.heappop(open_list)  # Pop the node with the lowest heuristic (best guess)
        closed_list.append(current)  # Mark the current node as visited

        if current == goal:  # Check if we reached the goal
            path = reconstruct_path(came_from, current)  # Reconstruct the path using the came_from dictionary
            return path, closed_list  # Return the path and visited nodes

        for neighbor, cost in graph[current]:  # Loop through neighbors of the current node
            if neighbor not in closed_list:  # Visit only unvisited neighbors
                heapq.heappush(open_list, (heuristic[neighbor], neighbor))  # Add neighbor with heuristic to open list
                came_from[neighbor] = current  # Record the path to this neighbor

    return None, closed_list  # If goal not found, return None and the visited nodes

# Function to reconstruct the final path from goal to start
def reconstruct_path(came_from, current):
    path = [current]  # Start with the goal node
    while current in came_from:  # Move backward using the came_from dictionary
        current = came_from[current]  # Get the parent of current node
        path.append(current)  # Add it to the path
    path.reverse()  # Reverse the list to get path from start to goal
    return path  # Return the final path
```

📊 **Input Data**

```
# Graph structure with edge costs
```

```
graph = {
    'S': [('A', 1), ('B', 1)],      # S connects to A and B
    'A': [('S', 1), ('G', 1)],      # A connects to S and G
    'B': [('S', 1), ('C', 1)],      # B connects to S and C
    'C': [('B', 1), ('D', 1)],      # C connects to B and D
    'D': [('C', 1), ('G', 1)],      # D connects to C and G
    'G': [('A', 1), ('D', 1)]       # G connects to A and D
}

# Heuristic values for each node
heuristic = {
    'S': 7,    # Estimated cost from S to G
    'A': 6,    # Estimated cost from A to G
    'B': 5,    # Estimated cost from B to G
    'C': 4,    # ...
    'D': 2,
    'G': 0     # Heuristic of goal is always 0
}
```

🚀 **Running the Search**

```
# Initial and goal states
start_node = 'S'
goal_node = 'G'

# Perform Best-First Search
path, closed_list = best_first_search(graph, heuristic, start_node, goal_node)

# Display results
print("Shortest Path:", path)       # Show final path
print("Closed List:", closed_list)  # Show visited nodes in order
```

🧠 Manual Solution (Step-by-Step)

Heuristic values:
S=7, A=6, B=5, C=4, D=2, G=0

Step-by-step Execution:

1. Start at **S** → Neighbors: A(6), B(5)
   Pick **B** (lowest heuristic 5)
2. From **B** → Neighbors: S, C(4)
   Pick **C**
3. From **C** → Neighbor: D(2)
   Pick **D**
4. From **D** → Neighbor: G(0)
   Pick **G** → Goal reached

Final Path:
S → B → C → D → G

Closed List (visited nodes):
['S', 'B', 'C', 'D', 'G']

Let me know if you want the same manual logic explained as a tree or animated step simulation for better visualization!